

SLOT 2

FLUTTER FUNDAMENTALS

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Basic UI Composition with Scaffold
- Summary & Setup Issues (môi trường, lỗi cấu hình, hiểu nhầm ban đầu)
- Dart Essentials

Mục tiêu tổng quát:

Giúp sinh viên **hiểu đúng tư duy Flutter**, **xây dựng được UI cơ bản với Scaffold**, **nắm nền tảng Dart để đọc – viết code Flutter**, và **tự phát hiện lỗi môi trường thường gặp**.

1 BASIC UI COMPOSITION WITH SCAFFOLD

(Xây dựng giao diện cơ bản với Scaffold)

Khái niệm cốt lõi cần hiểu

- Flutter xây dựng UI theo tư duy **Everything is a Widget**
- UI được tổ chức dưới dạng **Widget Tree**
- **Scaffold là khung giao diện chuẩn cho một màn hình (screen/page)** trong Flutter
- Mỗi màn hình thường tương ứng với **1 Scaffold**

Mục tiêu học tập (Sinh viên làm được gì?)

- Hiểu vai trò của Scaffold trong một ứng dụng Flutter
- Tạo được một màn hình Flutter đơn giản có:
 - AppBar
 - Body
 - Button / Icon cơ bản
- Biết cách ghép các widget đơn giản để tạo UI hoàn chỉnh

Ý nghĩa

Đây là **nền móng UI quan trọng nhất của Flutter**. Nếu sinh viên hiểu sai Scaffold, các module về layout, navigation và state sau này sẽ rất khó tiếp thu.

2 WIDGET TREE, LAYOUT & CĂN CHỈNH CƠ BẢN

(Hiểu cấu trúc UI và cách sắp xếp widget)

Khái niệm cốt lõi cần hiểu

- Widget trong Flutter có **quan hệ cha – con**
- Column, Row dùng để sắp xếp widget theo:
 - Chiều dọc
 - Chiều ngang
- mainAxisAlignment và crossAxisAlignment quyết định **cách căn chỉnh UI**

Mục tiêu học tập

- Đọc hiểu cấu trúc UI thông qua Widget Tree
- Sử dụng Column, Row, Center để sắp xếp UI cơ bản
- Biết vì sao **body chỉ nhận 1 widget** và cách xử lý bằng widget bọc

Ý nghĩa

Giúp sinh viên **không học Flutter theo kiểu thử – sai mù mờ**, mà hiểu rõ vì sao UI hiển thị như vậy.

3 BASIC UI COMPOSITION

(Ghép giao diện và quản lý khoảng cách)

Khái niệm cốt lõi cần hiểu

- UI Flutter được tạo bằng cách **ghép nhiều widget nhỏ**
- Khoảng cách UI được quản lý bằng:

- Padding
- SizedBox
- Không có padding mặc định như HTML/CSS

◆ Mục tiêu học tập

- Tạo được UI nhỏ như:
 - Login form đơn giản
 - Profile mini
- Biết so sánh UI có padding và không padding
- Nhận diện lỗi layout phổ biến (UI dính, lệch, khó nhìn)

◆ Ý nghĩa

Rèn tư duy **UI rõ ràng – dễ đọc – có khoảng thở**, tránh viết UI “đúng code nhưng xấu giao diện”.

4 WIDGET & STATE – LIÊN HỆ VỚI SCAFFOLD

(UI tĩnh và UI động)

◆ Khái niệm cốt lõi cần hiểu

- StatelessWidget: UI không thay đổi
- StatefulWidget: UI thay đổi theo dữ liệu
- `setState()` là cách Flutter biết **khi nào cần vẽ lại UI**

◆ Mục tiêu học tập

- Phân biệt khi nào cần StatelessWidget, khi nào cần StatefulWidget
- Hiểu vì sao thay đổi biến nhưng UI không cập nhật
- Sử dụng `setState()` đúng vị trí để cập nhật body của Scaffold

◆ Ý nghĩa

Đây là bước chuyển quan trọng từ **UI tĩnh sang UI có tương tác**, chuẩn bị cho các khái niệm state management sau này.

5 TÁCH WIDGET & CLEAN CODE CƠ BẢN

(Viết code dễ đọc – dễ bảo trì)

◆ Khái niệm cốt lõi cần hiểu

- Một widget không nên quá dài và phức tạp
- Tách widget giúp:
 - Dễ đọc code
 - Dễ tái sử dụng
 - Dễ sửa lỗi

◆ Mục tiêu học tập

- Biết khi nào cần tách widget
- Tạo được widget tái sử dụng đơn giản
- Hiểu cấu trúc thư mục cơ bản: `screens/`, `widgets/`

◆ Ý nghĩa

Giúp sinh viên **thoát khỏi thói quen viết code “một file tất cả”**, hướng đến tư duy dự án thực tế.

6 SUMMARY & SETUP ISSUES

(Môi trường, lỗi cấu hình và hiểu nhầm ban đầu)

◆ Khái niệm cốt lõi cần hiểu

- Không phải lỗi nào cũng do code
- Flutter phụ thuộc nhiều vào:
 - SDK
 - Emulator / thiết bị thật
 - IDE plugin

◆ Mục tiêu học tập

- Biết sử dụng `flutter doctor` để tự kiểm tra môi trường

- Phân biệt:
 - Lỗi do code
 - Lỗi do môi trường
- Hiểu khi nào Hot Reload không hoạt động

◆ Ý nghĩa

Giúp sinh viên **giảm tâm lý hoang mang**, tránh “debug mù” khi lỗi đến từ môi trường.

7 DART ESSENTIALS

(Nền tảng Dart để học Flutter hiệu quả)

◆ Khái niệm cốt lõi cần hiểu

- Dart là ngôn ngữ chính của Flutter
- Dart có:
 - Kiểu dữ liệu rõ ràng
 - Null Safety
 - Hỗ trợ async / await

◆ Mục tiêu học tập

- Đọc hiểu code Flutter cơ bản
- Sử dụng biến, hàm, điều kiện, vòng lặp trong Dart
- Hiểu null safety để tránh lỗi runtime
- Viết được class, enum đơn giản phục vụ UI

◆ Ý nghĩa

Dart không học để “thuần ngôn ngữ”, mà học để **viết UI Flutter đúng và an toàn**.

📖 NGUỒN TÀI LIỆU THAM KHẢO UY TÍN (DỄ TIẾP CẬN)

1. Flutter Official Documentation

👉 <https://docs.flutter.dev>

(Nguồn chính thống, nhiều ví dụ cơ bản)

2. Flutter YouTube – Official Channel

👉 Flutter Widget of the Week

(Video ngắn, dễ hiểu cho người mới)

3. Dart Official Documentation

👉 <https://dart.dev/guides>

(Giải thích Dart rõ ràng, có ví dụ)

4. Book:

Mastering Flutter Learn to develop Flutter apps for iOS, Android, desktop and web (Kevin Moore)

Flutter Apprentice (Raywenderlich / Kodeco)

(Phù hợp sinh viên mới – nhiều hình minh họa)

5. Fireship – Flutter in 100 Seconds (YouTube)

(Giúp sinh viên nắm bức tranh tổng thể nhanh)

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 TƯ DUY FLUTTER – EVERYTHING IS A WIDGET

(Cốt lõi tư duy, bắt buộc phải hiểu)

◆ Khái niệm quan trọng (BẮT BUỘC PHẢI HIỂU)

- **Widget là bản mô tả UI**, không phải là UI hiển thị thật
- Mọi thành phần trong Flutter (Text, Icon, Padding, khoảng trống) **đều là Widget**
- UI Flutter được xây dựng bằng cách **lồng widget vào nhau**

◆ Kỹ năng / thao tác phải THÀNH THẠO

- Nhận diện widget trong code Flutter
- Đọc và hiểu **widget tree đơn giản**
- Biết widget nào là **widget hiển thị**, widget nào là **widget bố cục**

◆ Lỗi thường gặp của sinh viên mới

- ❌ Nghĩ widget giống View cố định như Android XML
- ❌ Không hiểu widget bọc để làm gì → code rối, UI sai

Nguyên nhân: chưa hiểu Flutter render UI theo cây widget

◆ Best Practices (áp dụng ngay)

- Luôn đọc UI theo thứ tự **từ ngoài vào trong**
- Tự hỏi: *widget này đang bọc widget nào?*

📖 Ý nghĩa

Nếu không hiểu “Everything is a Widget”, sinh viên sẽ **học Flutter bằng cách đoán**, không thể đi xa hơn UI cơ bản.

2 WIDGET TREE & CẤU TRÚC UI

(Hiểu cấu trúc để không viết UI mù mờ)

◆ Khái niệm quan trọng

- Flutter UI là **cây widget**, có cha – con rõ ràng
- Widget cha ảnh hưởng trực tiếp đến cách hiển thị widget con

◆ Kỹ năng phải THÀNH THẠO

- Vẽ lại widget tree của một UI đơn giản
- Biết xác định widget cha khi UI hiển thị sai

◆ Lỗi thường gặp

- ❌ Đặt widget sai cấp → UI không căn chỉnh như mong muốn
- ❌ Dùng nhiều widget nhưng không biết widget nào ảnh hưởng layout

Nguyên nhân: không phân tích widget tree trước khi code

◆ Best Practices

- Khi UI sai → **nhìn widget tree trước, sửa code sau**
- Viết UI từ khung ngoài → chi tiết bên trong

📖 Ý nghĩa

Giúp sinh viên **kiểm soát UI**, thay vì thử – sai liên tục.

3 SCAFFOLD – KHUNG GIAO DIỆN BẮT BUỘC

(Trọng tâm lớn nhất của Module 2)

◆ Khái niệm quan trọng

- **Scaffold là khung giao diện chuẩn cho một màn hình Flutter**
- Gần như **mỗi screen = một Scaffold**

◆ Kỹ năng phải THÀNH THẠO

- Tạo Scaffold tối thiểu có AppBar và body
- Biết thêm:

- FloatingActionButton
- Drawer (ở mức nhận diện)
- Biết khi nào **bắt buộc phải dùng Scaffold**

❖ Lỗi thường gặp

- ❌ Không có AppBar → do thiếu Scaffold
- ❌ SnackBar / FAB không hoạt động

Nguyên nhân: chưa hiểu vai trò trung tâm của Scaffold

❖ Best Practices

- Mỗi screen → tạo 1 Scaffold riêng
- Không nhét nhiều màn hình vào cùng một Scaffold

📖 Ý nghĩa

Scaffold là “xương sống UI”. Hiểu sai Scaffold → **toàn bộ Flutter UI về sau sẽ rối.**

4 BODY CHỈ NHẬN 1 WIDGET – NGUYÊN LÝ BỌC

(Điểm sinh viên yếu hay vấp nhất)

❖ Khái niệm quan trọng

- body của Scaffold **chỉ nhận 1 widget**
- Muốn nhiều widget → **bắt buộc phải bọc**

❖ Kỹ năng phải THÀNH THẠO

- Sử dụng Column, Row, Center để bọc widget
- Hiểu vì sao Flutter yêu cầu 1 widget gốc

❖ Lỗi thường gặp

- ❌ Truyền nhiều widget trực tiếp vào body
- ❌ Không biết nên dùng Column hay Row

Nguyên nhân: quen tư duy HTML/XML nhiều phần tử ngang cấp

❖ Best Practices

- Luôn xác định: *widget gốc của body là gì?*
- Dùng Column mặc định khi chưa chắc

📖 Ý nghĩa

Nội dung này giúp sinh viên **chuyển tư duy từ UI khai báo sang UI có cấu trúc.**

5 COLUMN – ROW – CĂN CHỈNH UI

(Kỹ năng layout bắt buộc)

❖ Khái niệm quan trọng

- Column → sắp xếp dọc
- Row → sắp xếp ngang
- mainAxisAlignment & crossAxisAlignment quyết định căn chỉnh

❖ Kỹ năng phải THÀNH THẠO

- Căn giữa, căn trái/phải UI đơn giản
- Hiểu trục chính – trục phụ khi dùng Column/Row

❖ Lỗi thường gặp

- ❌ Căn sai trục → UI lệch
- ❌ Không hiểu vì sao đổi alignment không có tác dụng

Nguyên nhân: nhầm trục của Column và Row

❖ Best Practices

- Xác định **chiều chính trước, căn chỉnh sau**
- Không thử mò từng alignment

📖 Ý nghĩa

Đây là nền tảng cho mọi layout Flutter, kể cả layout nâng cao sau này.

6 PADDING – SIZEDBOX – QUẢN LÝ KHOẢNG CÁCH

(UI đúng nhưng phải đẹp)

◆ Khái niệm quan trọng

- Flutter **không có khoảng cách mặc định**
- Mọi khoảng trống phải tạo bằng widget

◆ Kỹ năng phải THÀNH THẠO

- Thêm Padding hợp lý cho UI
- Dùng SizedBox để tạo khoảng cách rõ ràng

◆ Lỗi thường gặp

- ❌ UI dính sát, khó nhìn
- ❌ Lạm dụng nhiều Padding lồng nhau

Nguyên nhân: chưa có tư duy bố cục UI

◆ Best Practices

- Dùng Padding ở cấp cao
- Dùng SizedBox cho khoảng cách nhỏ

🎓 Ý nghĩa

Giúp sinh viên **tạo UI có chất lượng**, không chỉ “chạy được”.

7 STATEFUL VS STATELESS – CẬP NHẬT UI

(Bước chuyển từ UI tĩnh sang UI động)

◆ Khái niệm quan trọng

- StatelessWidget → UI không đổi
- StatefulWidget → UI thay đổi
- `setState()` báo Flutter vẽ lại UI

◆ Kỹ năng phải THÀNH THẠO

- Biết khi nào cần StatefulWidget
- Gọi `setState()` đúng chỗ

◆ Lỗi thường gặp

- ❌ Thay đổi biến nhưng UI không đổi
- ❌ Dùng StatelessWidget cho UI có tương tác

Nguyên nhân: chưa hiểu cơ chế render của Flutter

◆ Best Practices

- UI có thay đổi → mặc định dùng StatefulWidget
- Mọi thay đổi UI → phải nằm trong `setState`

🎓 Ý nghĩa

Đây là nền móng cho toàn bộ state management sau này.

8 DART ESSENTIALS – DÙNG ĐỂ VIẾT FLUTTER

(Không học Dart lan man)

◆ Khái niệm quan trọng

- Dart là ngôn ngữ của Flutter
- Null safety giúp tránh crash

◆ Kỹ năng phải THÀNH THẠO

- Khai báo biến, hàm
- Sử dụng `if`, `for`
- Hiểu `?`, `!`, `??`

◆ Lỗi thường gặp

- ❌ Lỗi null nhưng không hiểu nguyên nhân
- ❌ Nhầm `var`, `final`, `const`

◆ Best Practices

- Khai báo kiểu rõ ràng
- Tránh dynamic khi mới học

🎓 Ý nghĩa

Dart là công cụ để viết UI Flutter an toàn, không phải để học OOP phức tạp lúc này.

9 SETUP & DEBUG CƠ BẢN

(Giảm stress cho sinh viên mới)

◆ Khái niệm quan trọng

- Không phải lỗi nào cũng do code
- flutter doctor là công cụ bắt buộc biết

◆ Kỹ năng phải THÀNH THẠO

- Chạy flutter doctor
- Phân biệt lỗi code vs lỗi môi trường

◆ Lỗi thường gặp

- ❌ Hot reload không chạy
- ❌ Emulator lỗi nhưng tưởng code sai

◆ Best Practices

- Kiểm tra môi trường trước khi sửa code
- Restart khi nghi ngờ lỗi lạ

🎓 Ý nghĩa

Giúp sinh viên **không bỏ cuộc sớm vì lỗi kỹ thuật**.

🔑 TỔNG KẾT TRỌNG TÂM MODULE 2

👉 Sau Module 2, sinh viên **BẮT BUỘC** phải:

- Hiểu Scaffold và widget tree
- Tạo được UI Flutter cơ bản, có cấu trúc
- Biết vì sao UI hiển thị đúng / sai
- Không hoảng khi gặp lỗi môi trường

III. ỨNG DỤNG THỰC TIỄN

1 MODULE 2 ĐƯỢC DÙNG NHƯ THẾ NÀO TRONG DỰ ÁN FLUTTER THỰC TẾ?

Góc nhìn thực tế:

Dù là MVP nhỏ, app sinh viên, hay app nội bộ, 80% màn hình trong giai đoạn đầu chỉ dùng kiến thức Module 2.

XÂY DỰNG UI CÓ CẤU TRÚC RÕ RÀNG

Áp dụng kỹ thuật:

- Scaffold → khung chuẩn cho từng màn hình
- AppBar, body, FloatingActionButton → bố cục nhất quán
- Column, Row, Padding → sắp xếp nội dung

Vai trò thực tế:

- Mỗi screen có cấu trúc rõ → dễ thêm chức năng sau này
- UI thống nhất → không “mỗi màn hình một kiểu”

Trong dự án thật:

Nếu sinh viên không vững Scaffold & layout cơ bản, UI sẽ rối ngay từ tuần đầu.

QUẢN LÝ DỮ LIỆU ĐƠN GIẢN (STATE CỤC BỘ)

Áp dụng kỹ thuật:

- StatefulWidget
- setState()
- Dart variables (bool, int, String)

Vai trò thực tế:

- Hiển thị / ẩn UI
- Cập nhật số lượng, trạng thái nút
- Form nhập liệu đơn giản

Trong MVP:

70–80% logic ban đầu chỉ cần setState, chưa cần Provider / Bloc.

ĐẢM BẢO APP CHẠY ỔN ĐỊNH (SETUP & MÔI TRƯỜNG)

Áp dụng kỹ thuật:

- flutter doctor
- Emulator / thiết bị thật
- Hiểu Hot Reload vs Hot Restart

Vai trò thực tế:

- Giảm thời gian debug vô ích
- Phân biệt lỗi code và lỗi môi trường

Trong dự án thật:

Rất nhiều “bug nghiêm trọng” thực chất là lỗi emulator hoặc SDK.

KẾT LUẬN NHÓM 1

Module 2 là nền tảng bắt buộc cho mọi dự án Flutter, dù:

- App nhỏ
- App demo
- Hay app thương mại giai đoạn đầu

2 CÁC TÌNH HUỐNG THỰC TẾ (PROJECT SCENARIOS)

TÌNH HUỐNG 1 – APP SINH VIÊN QUẢN LÝ CÁ NHÂN

Bối cảnh thực tế

- App quản lý:
 - Thời khóa biểu
 - Ghi chú cá nhân
- Dự án học kỳ của sinh viên

Vấn đề gặp phải

- UI rối, không đồng nhất
- Code dồn hết vào `main.dart`
- Bấm nút nhưng UI không thay đổi

Cách áp dụng Module 2

- Mỗi màn hình → 1 Scaffold
- StatelessWidget cho màn hình hiển thị tĩnh
- StatefulWidget cho:
 - Checkbox hoàn thành
 - Button thêm ghi chú
- `setState()` để cập nhật danh sách

Mục tiêu đạt được

- UI rõ ràng, dễ hiểu
- Code dễ chia nhỏ
- Sinh viên hiểu **vì sao UI thay đổi**

TÌNH HUỐNG 2 – MVP APP GIỚI THIỆU CÔNG TY

Bối cảnh thực tế

- App giới thiệu:
 - Công ty
 - Sản phẩm
 - Liên hệ
- Dùng demo cho khách hàng

Vấn đề gặp phải

- Mỗi màn hình layout khác nhau
- AppBar lúc có lúc không
- Lỗi hiển thị trên emulator khác nhau

Cách áp dụng Module 2

- Chuẩn hóa:
 - Mỗi screen = 1 Scaffold
- Dùng Dart enum & if để đổi nội dung hiển thị
- Kiểm tra `flutter doctor` trước khi demo

Mục tiêu đạt được

- UI đồng bộ
- App chạy ổn định trên nhiều thiết bị
- Demo mượt, không lỗi vặt

TÌNH HUỐNG 3 – APP NỘI BỘ NHỎ (INTERNAL TOOL)

Bối cảnh thực tế

- App nội bộ cho nhân viên:
 - Check-in
 - Ghi chú công việc
- Không yêu cầu phức tạp

Vấn đề gặp phải

- UI đơn giản nhưng khó bảo trì
- Logic UI lẫn lộn
- Thêm tính năng nhỏ mất nhiều thời gian

Cách áp dụng Module 2

- Tách widget nhỏ (button, card)
- StatefulWidget cho trạng thái check-in
- Dart function tách riêng logic xử lý

Mục tiêu đạt được

- Code gọn, dễ đọc
- Dễ thêm tính năng
- MVP triển khai nhanh

3 CASE STUDY – MVP ỨNG DỤNG QUẢN LÝ CÔNG VIỆC (TODO APP)

Bối cảnh dự án

- MVP quản lý công việc cá nhân
- Đội 1–2 người
- Thời gian phát triển ngắn

Vấn đề ban đầu

- UI ban đầu viết nhanh → rối
- Logic thêm/xóa task khó theo dõi
- Lỗi UI không cập nhật

Áp dụng Module 2

- Mỗi màn hình → Scaffold
- StatefulWidget quản lý danh sách task
- setState() khi thêm / xóa
- Dart List, for, if hiển thị task
- Kiểm tra môi trường trước mỗi buổi code

Kết quả đạt được

- UI rõ ràng, dễ dùng
- Code dễ đọc cho người mới
- MVP hoàn thành đúng thời hạn
- Dễ nâng cấp lên state management nâng cao

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

Điều QUAN TRỌNG NHẤT

- **Hiểu tư duy widget trước khi viết code**
- Không vội học thư viện nâng cao

✗ Sai lầm sinh viên hay mắc

- Nhảy thẳng vào Provider / Bloc
- Viết UI không có Scaffold
- Không kiểm tra môi trường
- Debug UI bằng cách “thử đại”

✓ Nguyên tắc cần duy trì lâu dài

- Mỗi màn hình → cấu trúc rõ ràng
- Chia nhỏ UI thành widget
- UI thay đổi → phải có state
- Kiểm tra môi trường sớm, thường xuyên

5 TỔNG KẾT SỰ PHẠM

**Module 2 không phải để tạo app phức tạp,
mà để tạo NỀN MÓNG ĐÚNG.**

Sinh viên học tốt Module 2 sẽ:

- Không sợ UI Flutter
- Không sợ state
- Không hoảng khi gặp lỗi
- Sẵn sàng bước sang:
 - Navigation
 - Layout nâng cao
 - State management

IV. MỘT SỐ NỘI DUNG CHI TIẾT

PHẦN A. BASIC UI COMPOSITION WITH SCAFFOLD

I. TƯ DUY NỀN TẢNG FLUTTER

1. Everything is a Widget

📖 Khái niệm

Trong Flutter:

MỌI THỨ HIỂN THỊ TRÊN MÀN HÌNH ĐỀU LÀ WIDGET

Bao gồm:

- Text
- Button
- Khoảng cách
- Màu nền
- Layout
- Màn hình

📖 Hiểu sai thường gặp

Hiểu sai	Hiểu đúng
Widget chỉ là Button	Widget là mọi thành phần UI
UI = XML	UI = Widget tree

👉 Flutter **không dùng XML, không dùng layout file.**

📖 Sinh viên thường hiểu sai

- ❌ Nghĩ widget = View cố định
- ❌ Nghĩ widget chỉ để hiển thị

👉 **Đúng:** widget có thể chỉ để **bọc, căn chỉnh, tạo khoảng cách**

📖 Liên hệ UI truyền thống

Flutter	Android XML	HTML/CSS
Widget	View	Element
Column	LinearLayout	flex-column
Padding	margin/padding	padding

🔥 **Học để làm gì?**

→ Để **đọc hiểu UI Flutter**, không code mò.

2. Cấu trúc cây Widget (Widget Tree)

📖 Khái niệm

- Widget Tree là **cấu trúc phân cấp cha – con** của UI.
- Mỗi widget:
 - Có cha
 - Có thể có con

📖 Nguyên tắc

- Widget ngoài **bao** widget trong
- UI càng phức tạp → cây càng sâu

Ví dụ:

```
MaterialApp
├── Scaffold
│   └── Column
│       ├── Text
│       └── Button
```

```
MaterialApp
├── Scaffold
│   └── Center
│       └── Text("Hello Flutter")
```

📖 Vì sao phải là cây?

- Flutter chỉ rebuild **nhánh bị thay đổi**
- Tối ưu hiệu năng

📖 Hiểu sai phổ biến

- ❌ Không biết widget nào ảnh hưởng widget nào
- ❌ UI sai nhưng không biết sửa từ đâu

📌 Học để làm gì?

→ Khi UI lỗi, **nhìn widget tree là biết lỗi ở đâu**

3. Vai trò của MaterialApp

📖 MaterialApp là gì?

- Là **widget gốc** của ứng dụng Flutter theo Material Design.
- Cung cấp:
 - Theme
 - Điều hướng (Navigator)
 - Ngôn ngữ
 - Scaffold context

📖 Nếu không có MaterialApp?

- AppBar hiển thị lỗi
- Theme không hoạt động
- Snackbar, Dialog lỗi

📌 Học để làm gì?

→ Để app Flutter **chạy đúng chuẩn Material Design**

📖 Lỗi thường gặp

❌ Bỏ MaterialApp → AppBar không hiển thị, UI lỗi

II. SCAFFOLD – KHUNG GIAO DIỆN CƠ BẢN

1. Scaffold là gì? Khi nào dùng?

- Scaffold là **khung giao diện chuẩn cho 1 màn hình Flutter**.
- Gần như **100% screen Flutter đều có Scaffold**
- Scaffold = bố cục xương sống của screen

📌 Quy tắc thực tế:

1 screen = 1 Scaffold

Khi nào cần dùng Scaffold?

Trường hợp	Có dùng Scaffold
Màn hình app	✓
Dialog nhỏ	✗
Widget con	✗

2. Các thành phần chính của Scaffold

- AppBar: tiêu đề màn hình
- body: nội dung chính
- FloatingActionButton: hành động nhanh
- Drawer: menu bên
- BottomNavigationBar: điều hướng đáy

📖 AppBar

- Thanh tiêu đề trên cùng
- Thường chứa: title, icon, menu

❗ Không có Scaffold → không có AppBar

Body

- Nội dung chính
- **Chỉ nhận 1 widget**

 Muốn nhiều widget → dùng Column / Row

FloatingActionButton

- Nút nổi
- Hành động chính (add, create...)

Drawer

- Menu trượt từ trái
- Điều hướng nhiều màn hình

BottomNavigationBar

- Thanh điều hướng phía dưới
- App nhiều tab

3. Mối quan hệ Scaffold & Screen

Screen	Scaffold
Home	1 Scaffold
Profile	1 Scaffold
Settings	1 Scaffold

 Không dùng 1 Scaffold cho nhiều màn hình

4. Nếu thiếu Scaffold thì sao?

-  Không có AppBar
-  FAB không hoạt động
-  Snackbar lỗi
-  UI thiếu padding chuẩn

 **Học để làm gì?**

→ Để xây UI **đúng kiến trúc Flutter ngay từ đầu**

III. XÂY DỰNG UI CƠ BẢN VỚI SCAFFOLD

1. Cấu trúc Scaffold tối thiểu

```
Scaffold(  
  appBar: AppBar(  
    title: Text('Home'),  
  ),  
  body: Center(  
    child: Text('Hello Flutter'),  
  ),  
);
```

2. Body chỉ nhận một Widget – vì sao?

- Flutter cần **1 node gốc**
- Muốn nhiều widget → **bọc bằng Column / Row**

 Sai:

```
body: Text("A"),  
body: Text("B"),
```



Đúng:

```
body: Column(
  children: [
    Text("A"),
    Text("B"),
  ],
)
```

3. Column – Row – Center

- Column: sắp xếp dọc
- Row: sắp xếp ngang
- Center: căn giữa

Căn chỉnh

- mainAxisAlignment: trục chính
- crossAxisAlignment: trục phụ



Học để làm gì?

→ Để UI không bị lệch, không overflow

IV. BASIC UI COMPOSITION (GHÉP GIAO DIỆN)

1. Widget cơ bản

- Text
- Icon
- ElevatedButton
- TextButton

2. Sắp xếp UI

- Column cho dọc
- Row cho ngang
- Lồng nhiều lớp

Ghép UI cơ bản

```
Column(
  children: [
    Icon(Icons.person),
    Text('Login'),
    ElevatedButton(
      onPressed: () {},
      child: Text('Submit'),
    ),
  ],
);
```

3. Quản lý khoảng cách

- Padding: khoảng cách bao quanh
- SizedBox: khoảng cách rõ ràng



Không padding → UI dính, xấu



Có padding → UI dễ đọc

4. Lỗi layout phổ biến

- RenderFlex overflow
- UI dính sát
- Column quá cao

🚩 Học để làm gì?

→ Tạo UI **đúng – đẹp – dễ mở rộng**

V. WIDGET & STATE (LIÊN HỆ SCAFFOLD)

StatelessWidget

- UI tĩnh
- UI không thay đổi

StatefulWidget

- Có state
- UI thay đổi theo dữ liệu

setState()

- Báo Flutter rebuild UI

❌ Đổi biến không setState → UI không cập nhật

🚩 Học để làm gì?

→ Tạo UI **có tương tác thật**

State ảnh hưởng body Scaffold

Khi state thay đổi → **body được rebuild**

VI. TÁCH WIDGET & CLEAN CODE

Khi nào tách widget?

- Widget dài > 30 dòng
- Tái sử dụng
- Dễ đọc – dễ sửa
- Khi widget > 100–150 dòng → nên tách
- Tách để:
 - Dễ đọc
 - Dễ test
 - Dễ bảo trì

Widget tái sử dụng

```
class MyButton extends StatelessWidget { ... }
```

Cấu trúc thư mục đề xuất

```
lib/  
├── screens/  
├── widgets/  
└── main.dart
```

🚩 Học để làm gì?

→ Chuẩn bị cho **dự án lớn**

VII. HOT RELOAD & DEBUG

Hot Reload

- Giữ state
- Nhanh
- Dừng khi sửa UI

Hot Restart

- Reset state
- Dừng khi UI lỗi

Debug cơ bản

- print()
- Error message
- Flutter Inspector

✗ Không hoạt động khi:

- Đổi constructor
- Đổi main()

VIII. LỖI THƯỜNG GẶP VỚI SCAFFOLD

Lỗi	Nguyên nhân	Cách fix
Không AppBar	Thiếu Scaffold	Bọc Scaffold
Overflow	Column quá cao	Scroll / Expanded
UI không đổi	Quên setState	Dùng đúng state
Import lỗi	Sai đường dẫn	Check import

IX. SETUP & MÔI TRƯỜNG

- flutter doctor
- Emulator / device thật
- SDK & plugin

flutter doctor

- Kiểm tra SDK
- Kiểm tra thiết bị

Emulator

- Android Emulator
- Physical device

SDK lỗi → UI lỗi

Môi trường sai → debug UI vô nghĩa

X. TỔNG KẾT PHẦN A

- Scaffold = trung tâm UI
- Widget Tree = tư duy Flutter
- Chuẩn bị cho:
 - Layout nâng cao
 - Navigation
 - State management

PHẦN B. DART ESSENTIALS

1. Dart là gì?

- Ngôn ngữ của Flutter
- Typed
- Compiled
- Null-safe



Học để làm gì?

→ Viết UI Flutter **an toàn, không crash**

2. Structure

```
void main() {  
  runApp(MyApp());  
}
```

- File .dart
- Hàm main()
- Import thư viện

3. Objectives

Sau bài học:

- Khai báo biến đúng
- Viết function
- Hiểu class
- Đọc hiểu code Flutter
- Viết logic UI
- Không sợ null & async

4. VARIABLES & TYPES

Built-in Types

- int, double
- String
- bool
- List, Set, Map
- dynamic, var
- final, const

Null Safety

Ký hiệu	Ý nghĩa
?	Có thể null
!	Tin chắc không null
??	Giá trị thay thế
late	Gán sau

👉 Tránh lỗi runtime

5. CONTROL FLOW

- if / else
- switch / case
- for, while, for-in

📌 Dùng để:

- Hiển thị UI điều kiện
- Xử lý danh sách

6. FUNCTIONS

- main()
- Anonymous function
- Arrow function
- Callback (button, gesture)

7. COMMENTS & DOC

- //
- /* */
- ///
- DartDoc

8. IMPORTS

- package
- relative
- pub.dev

9. CLASSES & OOP

- constructor
- named constructor
- mixins (nhận diện)

Constructors

- Default
- Named
- const

Mixins

- mixin
- with

10. ENUMS

- Trạng thái UI
- Mode hiển thị
- enum cơ bản
- enhanced enum

11. EXTENSIONS

- Làm code sạch hơn
- Đọc dễ hơn
- Mở rộng class

12. CONCURRENCY

- async / await
- Future
- Stream (nhận diện)

async / await

- Future

Streams

- Stream
- listen
- async*
- yield

13. EXCEPTIONS

- try – catch
- finally
- custom exception



KẾT LUẬN MODULE 2

**Module 2 không dạy bạn làm app phức tạp,
mà dạy bạn làm app ĐÚNG.**

Sau module này, sinh viên:

- Không mơ hồ Flutter
- Viết UI có cấu trúc
- Hiểu Dart đủ để đi tiếp
- Sẵn sàng cho module nâng cao

📖 TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

💠 Widget

Định nghĩa

Widget là **đơn vị nhỏ nhất để xây UI trong Flutter**.

Hiểu đúng

- Mọi thứ trong Flutter đều là widget
- Không có UI nào “không phải widget”

Ví dụ

Text, Button, Icon, Padding, Scaffold → đều là widget

💠 Everything is a Widget

Ý nghĩa

Flutter xây UI bằng cách **ghép các widget nhỏ thành widget lớn**.

Sai lầm phổ biến

Sinh viên nghĩ:

“Widget = Button / Text” ❌

Thực tế:

Layout, khoảng cách, màu sắc cũng là widget ✅

💠 Widget Tree

Định nghĩa

Widget Tree là **cấu trúc cây** thể hiện mối quan hệ cha – con giữa các widget.

Hiểu đúng

- UI Flutter **luôn có thứ tự lồng nhau**
- Widget ngoài bọc widget trong

Ví dụ

```
MaterialApp
├── Scaffold
│   └── Column
│       ├── Text
│       └── Button
```

💠 MaterialApp

Định nghĩa

Widget gốc cung cấp:

- Theme
- Điều hướng
- Material Design

Sai lầm phổ biến

- Bỏ MaterialApp → app lỗi UI, không có theme

II. SCAFFOLD & UI COMPOSITION

💠 Scaffold

Định nghĩa

Scaffold là **khung giao diện chuẩn cho 1 màn hình Flutter**.

Vai trò

- Quản lý AppBar
- Body
- FloatingActionButton
- Drawer

Hiểu đúng

👉 **Mỗi màn hình = 1 Scaffold (đa số trường hợp)**

◆ AppBar

Thanh tiêu đề phía trên màn hình.

Lỗi hay gặp

- AppBar không hiển thị → quên Scaffold

◆ Body

Khu vực **nội dung chính** của màn hình.

Nguyên tắc quan trọng

👉 body **chỉ nhận 1 widget duy nhất**

➡ Muốn nhiều widget → dùng Column / Row

◆ Column

Widget sắp xếp con **theo chiều dọc**.

Dùng khi

- Form
- Danh sách nút
- Text xếp từ trên xuống

◆ Row

Widget sắp xếp con **theo chiều ngang**.

Dùng khi

- Icon + Text
- Button đặt cạnh nhau

◆ Padding

Widget tạo **khoảng cách bên trong**.

Hiểu sai phổ biến

- Padding ≠ margin (CSS)

◆ SizedBox

Widget tạo **khoảng cách cố định** (rộng / cao).

III. WIDGET & STATE

◆ StatelessWidget

Định nghĩa

Widget **không thay đổi UI** sau khi vẽ.

Dùng khi

- Màn hình tĩnh
- UI chỉ hiển thị

◆ StatefulWidget

Định nghĩa

Widget có **state (dữ liệu)** và UI có thể thay đổi.

Dùng khi

- Có input
- Có danh sách
- Có tương tác người dùng

◆ State

Định nghĩa

State là dữ liệu ảnh hưởng trực tiếp đến UI.

Ví dụ

- Biến bool
- List
- Giá trị TextField

◆ setState()

Định nghĩa

Hàm thông báo cho Flutter:

“Dữ liệu đã thay đổi, hãy vẽ lại UI”

Sai lầm phổ biến

- Quên setState → UI không cập nhật

IV. HOT RELOAD & DEBUG

◆ Hot Reload

Định nghĩa

Cập nhật UI **ngay lập tức** khi sửa code.

Không áp dụng khi

- Sửa main()
- Sửa cấu trúc widget gốc

◆ Hot Restart

Định nghĩa

Restart app nhưng **nhANH hơn chạy lại từ đầu**.

◆ Debug

Định nghĩa

Quá trình tìm và sửa lỗi.

Công cụ

- print()
- Error message
- Flutter Inspector

V. DART ESSENTIALS – KIẾN THỨC NỀN

◆ Dart

Ngôn ngữ lập trình dùng trong Flutter.

◆ Variable (Biến)

Nơi lưu trữ dữ liệu.

```
int age = 20;
```

◆ Type (Kiểu dữ liệu)

Xác định loại dữ liệu.

Type	Ví dụ
int	1, 2
String	"Hello"
bool	true / false
List	[]
Map	{}

◆ final

Biến **chỉ gán 1 lần**, nhưng giá trị bên trong có thể đổi.

◆ const

Giá trị **bất biến hoàn toàn**, xác định lúc compile.

◆ Null Safety

Cơ chế **bắt lỗi null từ lúc viết code**.

`String? name; // có thể null`

◆ Function

Khối code thực hiện 1 nhiệm vụ.

◆ Control Flow

Điều khiển luồng chạy chương trình.

- if / else
- for
- switch

◆ Class

Khuôn mẫu tạo object.

◆ Constructor

Hàm khởi tạo object.

◆ async / await

Xử lý tác vụ mất thời gian (API, delay).

◆ Exception

Lỗi xảy ra khi chạy app.

📖 LỜI KHUYÊN CHO SINH VIÊN

👉 Nếu **chưa giải thích được thuật ngữ bằng lời của mình**

➡ Nghĩa là **chưa hiểu thật**

👉 Hãy:

- Đọc lại glossary
- Tìm thuật ngữ trong code
- Nói lại bằng ngôn ngữ đơn giản

V. KINH NGHIỆM THỰC TIỄN, TIPS

I. BASIC UI COMPOSITION WITH SCAFFOLD

1. Kỹ thuật: Scaffold – Khung giao diện trung tâm

✓ Kinh nghiệm thực tế khi dùng trong dự án

- **Scaffold được dùng cho 100% các màn hình chính:**
 - Home, Login, Profile, Settings, Dashboard...
- Trong dự án thật:
 - Mỗi screen = 1 Scaffold
 - Scaffold giúp UI có cấu trúc rõ ràng & thống nhất
- **Tình huống thực tế**
 - 📱 App quản lý sinh viên
 - Màn hình Home cần: AppBar + nội dung + FAB thêm sinh viên
 - 📌 Scaffold là lựa chọn bắt buộc vì:
 - Quản lý AppBar
 - Hỗ trợ FAB, Drawer, BottomNavigationBar

⚠ Lỗi thường gặp

✗ Lỗi 1: Không dùng Scaffold → UI “lạ”, thiếu AppBar

- 🔍 Nguyên nhân: Chưa hiểu Scaffold là “khung màn hình”
- 🛠 Ví dụ:

```
return Center(child: Text("Home"));
```
- ✂ Cách khắc phục:

```
return Scaffold(  
  appBar: AppBar(title: Text("Home")),  
  body: Center(child: Text("Home")),  
);
```
- 🚫 Phòng tránh:
 - Luôn hỏi: “Màn hình này có phải 1 screen không?”
 - Nếu có → dùng Scaffold

💡 Tips & Tricks

- Tạo **template Scaffold** sẵn cho screen mới
- Luôn viết Scaffold theo thứ tự:
appBar → body → floatingActionButton

🛡 Best Practices

- ✅ 1 screen = 1 Scaffold
- ✗ Không lồng Scaffold trong Scaffold
- ✅ Scaffold chỉ dùng cho UI cấp màn hình

🔄 Liên hệ thực tế

- App MVP startup:
 - Khi bỏ Scaffold → mất Snackbar, Drawer
 - Khi thêm lại → UI ổn định, code dễ đọc hơn

🎓 Bài học rút ra

- **Scaffold = nền móng UI**
- Không có Scaffold → app “trông như chưa hoàn chỉnh”

II. WIDGET TREE & EVERYTHING IS A WIDGET

2. Kỹ thuật: Widget Tree – Everything is a Widget

✓ Kinh nghiệm thực tế

- Flutter **render UI theo cây Widget**
- Mọi thứ đều là widget:
 - Padding, Text, Icon, Button, Scaffold
- Tình huống thực tế**
 - UI bị lỗi spacing
 - Nguyên nhân: đặt Padding sai vị trí trong cây

⚠ Lỗi thường gặp

✗ UI không đúng bố cục dù code “có vẻ đúng”

- 🔍 Nguyên nhân:
 - Không hình dung được widget cha – con

- 🔧 Ví dụ:

```
Column(  
  children: [  
    Text("A"),  
    Padding(  
      padding: EdgeInsets.all(16),  
    ),  
    Text("B"),  
  ],  
);
```

- ✂ Cách khắc phục:

```
Padding(  
  padding: EdgeInsets.all(16),  
  child: Text("B"),  
);
```

💡 Tips

- Luôn **vẽ widget tree ra giấy**
- Đọc code từ ngoài → vào trong

🛡 Best Practices

- UI sai → kiểm tra **cấu trúc cây trước**, không debug logic vội
- Chia widget nhỏ để cây rõ ràng

🎓 Bài học rút ra

- Không hiểu Widget Tree → **không debug được UI**

III. STATELESS & STATEFUL WIDGET + setState

3. Kỹ thuật: StatelessWidget / StatefulWidget

✓ Kinh nghiệm thực tế

- StatelessWidget:**
 - UI tĩnh (Text, Icon, layout)
- StatefulWidget:**
 - UI thay đổi theo dữ liệu
- Tình huống thực tế**

- Nút “Like” → đổi icon
- Cần StatefulWidget + setState

⚠ Lỗi thường gặp

❌ UI không cập nhật

- 🔍 Nguyên nhân:
 - Quên setState
 - Đặt biến ngoài State
- 🖋 Ví dụ sai:


```
int count = 0;
```
- ✂ Cách đúng:


```
setState(() {
  count++;
});
```

💡 Tips

- UI đổi → kiểm tra:
 1. Có StatefulWidget chưa?
 2. Có setState chưa?

🛡 Best Practices

- State nhỏ → dùng setState
- Không lạm dụng StatefulWidget

🎓 Bài học rút ra

- **Không có setState → UI không sống**

IV. UI COMPOSITION – Column, Row, Center

💠 4. Kỹ thuật: Column / Row / Center

✅ Kinh nghiệm thực tế

- Dùng để **xếp layout**
- 90% UI cơ bản dùng 3 widget này
- **Tình huống**
 - Form Login → Column + Padding

⚠ Lỗi thường gặp

❌ RenderFlex overflow

- 🔍 Nguyên nhân:
 - Quá nhiều widget trong Column
- ✂ Cách khắc phục:
 - Dùng SingleChildScrollView

💡 Tips

- UI dài → nghĩ đến scroll sớm
- Test trên màn hình nhỏ

🛡 Best Practices

- Không nhồi quá nhiều widget vào 1 Column
- Chia widget con

Bài học rút ra

- Layout sai thường do **thiếu tư duy không gian**

V. HOT RELOAD & DEBUG

5. Kỹ thuật: Hot Reload

Kinh nghiệm thực tế

- Hot Reload **chỉ reload UI**
- Không reload:
 - main()
 - initState()

Lỗi thường gặp

- UI không đổi dù code đổi
- 🔍 Nguyên nhân: cần Hot Restart

Tips

- UI layout → Hot Reload
- Logic khởi tạo → Hot Restart

Bài học rút ra

- Đừng tin Hot Reload 100%

VI. SETUP & MÔI TRƯỜNG

6. Kỹ thuật: Flutter Setup

Kinh nghiệm thực tế

- Nhiều lỗi **không phải do code**
- flutter doctor là công cụ bắt buộc

Lỗi thường gặp

- Emulator chạy chậm
- SDK mismatch

Best Practices

- Check flutter doctor mỗi khi lỗi lạ
- Không sửa code khi môi trường chưa ổn

Bài học rút ra

- **Môi trường sai → code đúng cũng lỗi**

VII. DART ESSENTIALS (TỔNG HỢP)

7. Kỹ thuật: Dart cơ bản

Kinh nghiệm thực tế

- Dart quyết định:
 - UI hiển thị gì
 - UI hiển thị khi nào

Lỗi phổ biến

- dynamic quá nhiều
- null safety sai

Best Practices

- Dùng type rõ ràng
- Viết function nhỏ, dễ test

Bài học rút ra

- Flutter UI mạnh hay yếu → **phụ thuộc Dart**

TỔNG KẾT CHO SINH VIÊN

Nếu chỉ nhớ 5 điều:

1. Scaffold = khung màn hình
2. UI là widget tree
3. UI đổi → cần state + setState
4. Layout sai → do cấu trúc, không phải Flutter lỗi
5. Lỗi lạ → kiểm tra môi trường trước

Học Module 2 không để làm app đẹp, mà để làm app **ĐÚNG & CHẠY ỔN**.

EXERCISE

DỰ ÁN THỰC HÀNH 1

1 TỔNG QUAN DỰ ÁN

◆ Tên dự án

My Daily Planner App

◆ Ứng dụng này làm gì?

Đây là một ứng dụng Flutter **rất đơn giản**, cho phép:

- Hiển thị danh sách công việc
- Nhấn nút  để thêm công việc mới
- Mỗi lần nhấn nút, giao diện **tự cập nhật**

❌ **Không có database**

❌ **Không đăng nhập**

❌ **Không Provider / Bloc**

👉 Mục tiêu là **HIỂU CÁCH FLUTTER VỀ GIAO DIỆN & CẬP NHẬT UI**

◆ Sinh viên sẽ học được gì sau dự án?

Sau khi làm xong dự án, sinh viên sẽ:

- Biết **tạo project Flutter từ con số 0**
- Hiểu **MaterialApp** → **Scaffold** → **Widget Tree**
- Phân biệt khi nào dùng:
 - StatelessWidget
 - StatefulWidget
- Hiểu **setState** dùng để làm gì
- Tự kiểm tra được:
“Mình đã làm đúng hay chưa?”

◆ Đối tượng phù hợp

- Sinh viên **mới học Flutter**
- Sinh viên **chưa quen Android Studio**
- Sinh viên **còn sợ code Flutter**

2 CHỨC NĂNG ỨNG DỤNG (RẤT CỤ THỂ)

STT	Chức năng	Giải thích
1	Hiển thị tiêu đề	AppBar
2	Hiển thị danh sách task	Column + Text
3	Thêm task mới	FloatingActionButton
4	Cập nhật giao diện	setState

👉 Mỗi chức năng = **1 kiến thức Flutter cơ bản**

3 CÔNG NGHỆ & CÔNG CỤ (CHỈ NHỮNG GÌ CẦN)

- Flutter SDK (đã cài)
- Android Studio
- Android Emulator
- Dart
- Material Design

4 CẤU TRÚC THƯ MỤC (LÀM RÕ TỪNG THƯ MỤC)

Sau khi hoàn thành, thư mục lib/ sẽ có dạng:

```
lib/  
├── main.dart           // File chạy đầu tiên  
├── screens/  
│   └── home_screen.dart // Màn hình chính  
└── widgets/  
    └── task_item.dart   // Widget hiển thị 1 task
```

Quy tắc nhớ cho sinh viên

- main.dart → KHỞI ĐỘNG APP
- screens/ → MÀN HÌNH
- widgets/ → UI NHỎ TÁI SỬ DỤNG

5 TOÀN BỘ QUY TRÌNH (ĐỌC TRƯỚC – CHƯA LÀM)

1. Tạo project Flutter
2. Chạy app mặc định
3. Sửa main.dart
4. Tạo HomeScreen
5. Thêm Scaffold
6. Thêm AppBar
7. Tạo danh sách task
8. Hiển thị task
9. Thêm nút +
10. Dùng setState
11. Tách widget
12. Kiểm tra hoàn chỉnh

Bây giờ bắt đầu làm TỪNG BƯỚC

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (CẦM TAY CHỈ VIỆC)

BƯỚC 1: TẠO PROJECT FLUTTER

Mục tiêu

Tạo project Flutter **chạy được** trên máy.

Thao tác chi tiết

1. Mở **Android Studio**
2. Chọn **New Flutter Project**
3. Chọn **Flutter Application**
4. Nhấn **Next**
5. Nhập:
 - Project name: my_daily_planner
6. Nhấn **Finish**
7. Đợi Android Studio tạo project (1–2 phút)

CHECKPOINT – KIỂM TRA ĐÃ XONG CHƯA?

- ✓ Thấy file main.dart
- ✓ Thấy nút **Run**  trên thanh công cụ
- ✗ Nếu không thấy → hỏi giảng viên

◆ BƯỚC 2: CHẠY APP MẶC ĐỊNH

🎯 Mục tiêu

Đảm bảo Flutter chạy được.

🔧 Thao tác

1. Bật **Android Emulator**
2. Nhấn **Run** ▶️
3. Đợi app chạy

✅ CHECKPOINT

- ✅ Thấy app Flutter có **counter** + **nút** +
- ✅ Không có lỗi đỏ

◆ BƯỚC 3: DỌN DẸP main.dart

📁 Mở file: lib/**main.dart**

👉 Xóa toàn bộ code cũ

📄 Dán code sau

Thao tác: Mở file lib/main.dart, xóa hết code cũ và dán đoạn code chuẩn bị sẵn này vào.

```
import 'package:flutter/material.dart';
// Lưu ý: Dòng dưới sẽ báo đỏ cho đến khi bạn hoàn thành Bước 4
import 'screens/home_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false, // Tắt chữ Debug ở góc màn hình
      title: 'Daily Planner',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}
```

✅ CHECKPOINT

- ✅ Không có gạch đỏ
- ✅ Không lỗi import
- ⚠️ Nếu lỗi → **chưa tạo HomeScreen**

✅ **CHECKPOINT:** File main.dart sẽ báo lỗi ở dòng import 'screens/home_screen.dart';. Điều này là bình thường vì chúng ta chưa tạo file đó.

◆ BƯỚC 4: TẠO HomeScreen

🔧 Thao tác

1. Chuột phải lib/
2. New → Directory → đặt tên screens
3. Chuột phải screens
4. New → Dart File → home_screen.dart

👉 **Viết code (Khung cơ bản):** Dán đoạn code dưới đây để tạo cấu trúc StatefulWidget hoàn chỉnh.

```
import 'package:flutter/material.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // Dữ liệu sẽ nằm ở đây (Bước 6)

  @override
  Widget build(BuildContext context) {
    // Giao diện sẽ nằm ở đây (Bước 5)
    return const Scaffold(
      body: Center(child: Text('Initial Screen')),
    );
  }
}
```

✅ CHECKPOINT

- ✅ Không lỗi
- ✅ Android Studio tự gợi ý _HomeScreenState

◆ BƯỚC 5: THÊM Scaffold + AppBar

Thao tác: Cập nhật hàm build bên trong class _HomeScreenState để hiển thị khung ứng dụng.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('My Daily Planner'),
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    ),
    body: const Center(
      child: Text('No tasks yet'),
    ),
  );
}
```

✅ CHECKPOINT

- ✅ App có thanh trên cùng
- ✅ Có chữ **My Daily Planner**

◆ BƯỚC 6: TẠO DANH SÁCH TASK

```
List<String> tasks = [];
```

🚧 **Đặt dòng này ngay dưới class State**

Để tránh đặt sai vị trí biến `tasks` và hàm `setState`, hãy chú ý vị trí trong class:

```
class _HomeScreenState extends State<HomeScreen> {
  // BƯỚC 6: Khai báo danh sách ngay tại đây
  List<String> tasks = [];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('My Daily Planner')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: tasks.map((task) => Text(task)).toList(),
        ),
      ),
      // BƯỚC 8: Thêm nút bấm
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Lệnh setState bắt buộc phải bọc quanh việc thay đổi dữ liệu
          setState(() {
            tasks.add('Task thứ ${tasks.length + 1}!');
          });
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

✓ CHECKPOINT

- ✓ Không lỗi
- ✓ Chưa thấy gì thay đổi (ĐÚNG)

◆ BƯỚC 7: HIỂN THỊ DANH SÁCH TASK

Thao tác: Thay thế phần `body` cũ trong file `lib/screens/home_screen.dart` bằng đoạn code sử dụng `Column` để duyệt danh sách.

```
body: Padding(
  padding: const EdgeInsets.all(16.0),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start, // Căn lề trái cho các task
    children: tasks.map((task) {
      return Text(task, style: const TextStyle(fontSize: 18));
    }).toList(),
  ),
),
```

✓ CHECKPOINT

- App vẫn chạy bình thường.
- Màn hình trắng (hoặc chỉ hiện tiêu đề) vì danh sách `tasks` hiện đang rỗng `[]`.

◆ BƯỚC 8: THÊM NÚT BẤM VÀ CẬP NHẬT TRẠNG THÁI

Thao tác: Thêm `floatingActionButton` vào trong `Scaffold` (nằm sau phần `body`).

```
floatingActionButton: FloatingActionButton(  
  onPressed: () {  
    setState(() {  
      // Thêm một task mới vào danh sách mỗi khi bấm nút  
      tasks.add('Công việc số ${tasks.length + 1}');  
    });  
  },  
  tooltip: 'Thêm công việc',  
  child: const Icon(Icons.add),  
)
```

✓ CHECKPOINT

- Bấm nút +: Một dòng chữ mới xuất hiện ngay lập tức trên màn hình.
- **Giải thích:** Nếu thiếu `setState`, danh sách `tasks` vẫn tăng nhưng Flutter sẽ không "vẽ" lại màn hình

◆ BƯỚC 9: TÁCH WIDGET (TẠO FILE `task_item.dart`)

📁 `widgets/task_item.dart`

Mục tiêu: Giúp code sạch hơn bằng cách tách một dòng task thành một Widget riêng.

Thao tác:

1. Chuột phải vào `lib/` -> **New** -> **Directory** -> đặt tên là `widgets`.
2. Chuột phải vào `widgets/` -> **New** -> **Dart File** -> đặt tên là `task_item.dart`.
3. Dán đoạn code sau:

```
import 'package:flutter/material.dart';  
  
class TaskItem extends StatelessWidget {  
  final String title;  
  
  const TaskItem({super.key, required this.title});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      width: double.infinity,  
      margin: const EdgeInsets.only(bottom: 10),  
      padding: const EdgeInsets.all(12),  
      decoration: BoxDecoration(  
        color: Colors.blue.withOpacity(0.1),  
        borderRadius: BorderRadius.circular(8),  
      ),  
      child: Text(  
        title,  
        style: const TextStyle(fontSize: 16, fontWeight: FontWeight.w500),  
      ),  
    );  
  }  
}
```

✓ CHECKPOINT

- ✓ Không lỗi
- ✓ Có thể import được

📦 BƯỚC 10: DỪNG TaskItem

Thao tác: Quay lại file `lib/screens/home_screen.dart`.

1. Ở trên cùng file, thêm dòng: `import '../widgets/task_item.dart';`
2. Thay đổi phần `children` trong `Column` ở Bước 7:

Dart

```
// Thay thế đoạn Text(task) cũ bằng Widget mới
children: tasks.map((task) {
  return TaskItem(title: task); // Sử dụng Widget đã tách
}).toList(),
```

✓ CHECKPOINT CUỐI

- App hiển thị danh sách task đẹp hơn (có khung màu xanh, bo góc).
- Nhấn nút +: Các "thẻ" task mới xuất hiện mượt mà.
- Không có lỗi đỏ trong cửa sổ **Dart Analysis**.

- ✓ App chạy ổn
- ✓ UI gọn gàng
- ✓ Không crash

🎉 HOÀN THÀNH DỰ ÁN

7 SAU DỰ ÁN, SINH VIÊN ĐÃ LÀM ĐƯỢC GÌ?

- Tự tạo app Flutter
- Hiểu cách UI được vẽ
- Biết `setState` → **UI cập nhật**

Flutter Setup & Workflow Practice (Flutter Fundamentals – Module 2)

1 TỔNG QUAN DỰ ÁN

Tên dự án

Flutter Environment & Workflow Lab

Mô tả ngắn gọn

Đây là **một dự án thực hành KHÔNG TẬP TRUNG CODE**, mà tập trung vào:

- Cài đặt
- Kiểm tra
- Chạy
- Sửa lỗi
- Quan sát hành vi Flutter khi phát triển app

 Sinh viên **làm chủ công cụ trước khi làm app lớn**

Mục tiêu học tập

Sau dự án này, sinh viên:

- Biết **Flutter đang dùng SDK nào**
- Biết **lỗi môi trường xuất phát từ đâu**
- Biết **khi nào Hot Reload có tác dụng – khi nào không**
- Biết **debug lỗi UI cơ bản**

Đối tượng phù hợp

- Sinh viên **mới học Flutter**
- Sinh viên **hay gặp lỗi nhưng không hiểu lý do**
- Sinh viên **chưa quen Android Studio + Emulator**

2 CÁC NỘI DUNG SẼ THỰC HÀNH

Nội dung	Mục đích
Android Studio	Làm quen giao diện & công cụ
Emulator	Hiểu máy ảo hoạt động ra sao
flutter doctor	Kiểm tra & đọc lỗi môi trường
Hot Reload	Tăng tốc khi code
Debug cơ bản	Tìm lỗi UI / logic đơn giản

3 CÔNG NGHỆ & CÔNG CỤ

- Flutter SDK
- Dart
- Android Studio
- Android Emulator
- Terminal (trong Android Studio)

4 CẤU TRÚC DỰ ÁN SỬ DỤNG

DỪNG LẠI PROJECT 1 – KHÔNG TẠO MỚI

```
my_daily_planner/  
├── lib/  
├── android/  
└── pubspec.yaml
```

Lý do:

Sinh viên thường:

“Code chạy hôm qua – hôm nay lỗi”

 Dự án này giúp **biết cách xử lý tình huống đó**

5 TOÀN BỘ QUY TRÌNH THỰC HÀNH

1. Kiểm tra Android Studio
2. Kiểm tra Flutter SDK
3. Dùng flutter doctor
4. Bật Emulator đúng cách
5. Chạy app từ terminal
6. Dùng Hot Reload
7. Dùng Hot Restart
8. Debug lỗi UI
9. Đọc log lỗi
10. Tổng kết workflow đúng

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC (CẦM TAY CHỈ VIỆC)

BƯỚC 1: KIỂM TRA ANDROID STUDIO PLUGINS

Mục tiêu

Đảm bảo IDE có "thấu hiểu" được ngôn ngữ Dart và khung Flutter. Android Studio **đã sẵn sàng cho Flutter**

Thao tác chi tiết

1. Mở **Android Studio**.
2. Vào **File** → **Settings** (Windows) hoặc **Android Studio** → **Settings** (macOS).
3. Chọn mục **Plugins** ở cột bên trái.
4. Chuyển sang tab **Installed** và kiểm tra xem đã có **Flutter** và **Dart** chưa.

CHECKPOINT

Nếu thấy chữ **Installed**, bạn đã sẵn sàng. Nếu chưa, hãy sang tab **Marketplace**, tìm "Flutter", cài đặt và **Restart IDE**

 Flutter plugin: **Installed**

 Dart plugin: **Installed**

 Nếu chưa có → cài plugin → restart Android Studio

BƯỚC 2: KIỂM TRA FLUTTER SDK (XÁC THỰC ĐƯỜNG DẪN FLUTTER SDK)

Mục tiêu: Kiểm tra xem Android Studio có trỏ đúng vào bộ máy của Flutter không. **Thao tác:**

1. .

Thao tác

1. Vẫn trong bảng **File** → **Settings**, tìm mục **Languages & Frameworks** → **Flutter**.
2. Kiểm tra dòng **Flutter SDK path**
3. Xem đường dẫn Flutter SDK

✓ CHECKPOINT

✓ Đường dẫn phải trỏ đúng vào thư mục bạn đã giải nén Flutter (ví dụ: C:\flutter hoặc D:\src\flutter).
Nếu trống, nhấn dấu ... để chọn lại thư mục đó.

⚠ Nếu trống → Flutter chưa được setup đúng, nhấn dấu ... để chọn lại thư mục đó.

◆ BƯỚC 3: CHẠY flutter doctor

Mục tiêu

Kiểm tra **toàn bộ môi trường Flutter**. Để Flutter tự "khám bệnh" cho chính nó.

Thao tác

1. Mở tab **Terminal** ở cạnh dưới màn hình Android Studio.
2. Gõ lệnh: `flutter doctor` và nhấn Enter.
3. **Lưu ý quan trọng:** Nếu thấy lỗi `Android licenses not accepted`, hãy chạy tiếp lệnh:
`flutter doctor --android-licenses` và gõ y liên tục cho đến khi kết thúc.

Kết quả mong muốn (ví dụ)

Phải có dấu [✓] ở ít nhất 3 mục đầu tiên (Flutter, Android toolchain, Android Studio).

```
[✓] Flutter
[✓] Android toolchain
[✓] Android Studio
[✓] Connected device
```

✓ CHECKPOINT

✓ Có dấu ✓ ở Flutter & Android toolchain

⚠ Có ⚠ → đọc kỹ dòng phía dưới

🔴 **Quy tắc cho sinh viên**

✗ Không hoảng

✓ Đọc từ trên xuống

✓ Sửa từng lỗi MỘT

◆ BƯỚC 4: BẬT ANDROID EMULATOR ĐÚNG CÁCH

Thao tác

1. Android Studio
2. Nhấn biểu tượng **Device Manager**
3. Chọn 1 thiết bị (Pixel)
4. Nhấn  **Play**
5. Đợi cho đến khi máy ảo hiện màn hình chính của Android (giống điện thoại thật)

✓ CHECKPOINT

- ✓ Emulator hiện màn hình Android
- ✓ Có nút Home – Back – Recent
- ✗ Emulator đen màn hình → chờ thêm / restart

◆ BƯỚC 5: CHẠY APP BẰNG TERMINAL

Mục tiêu: Hiểu cách app được đóng gói và đẩy vào máy ảo mà không dùng nút Play.

✚ KHÔNG dùng nút Run – dùng lệnh

✂ Thao tác

1. Trong **Terminal**, gõ: `flutter run`.
2. Nếu máy hỏi chọn thiết bị, hãy gõ số thứ tự tương ứng với máy ảo đang bật

✓ CHECKPOINT

✓ Chờ cho đến khi thấy dòng `Running Gradle task 'assembleDebug'...` kết thúc và app hiện lên trên máy ảo. Terminal hiện:
`Running Gradle task...`

✓ App chạy trên Emulator

✚ Ý nghĩa

Khi gặp lỗi CI/CD sau này → vẫn chạy được

◆ BƯỚC 6: THỰC HÀNH HOT RELOAD

Mục tiêu: Thấy sự thay đổi giao diện trong chưa đầy 1 giây

✂ Thao tác

1. Mở file `lib/screens/home_screen.dart`.
 2. Tìm dòng `title: const Text('My Daily Planner')`.
 3. Đổi thành: `title: const Text('Flutter is Awesome!')`.
 4. Nhấn **Ctrl + S** (Windows) hoặc **Cmd + S** (macOS). Hot Reload'
-
3. Nhấn:
 - **Ctrl + S**

✓ CHECKPOINT Tiêu đề trên máy ảo đổi ngay lập tức mà app không hề khởi động lại.

✓ UI đổi NGAY

✓ Không restart app

✚ Giải thích

Flutter chỉ rebuild widget tree

◆ BƯỚC 7: SO SÁNH HOT RESTART

Mục tiêu: Hiểu khi nào dữ liệu bị mất (Reset State).

✂ Thao tác

1. Nhấn nút + để thêm vài task vào danh sách.
2. Tại tab **Terminal**, nhấn phím **Shift + R** (viết hoa) hoặc nút **Hot Restart** (biểu tượng mũi tên xoay tròn trên thanh công cụ).

1. Nhấn:
 - R trong terminal hoặc
 - Hot Restart button

✓ CHECKPOINT

- ✓ App chạy lại từ đầu
- ✓ State bị reset

📌 So sánh cho sinh viên

Hot Reload	Hot Restart
Nhanh	Chậm hơn
Giữ state	Mất state

📌 BƯỚC 8: DEBUG LỖI UI CƠ BẢN

Mục tiêu: Làm quen với "màn hình đỏ" và tìm nơi báo lỗi. **Thao tác:**

1. Trong hàm `build` của `HomeScreen`, hãy thử thêm dòng này vào đầu: `throw Exception('Lỗi giả định để học Debug');`
2. Lưu lại để app báo lỗi.
3. Mở tab **Run** hoặc **Debug Console** ở dưới cùng.
4. Tìm dòng chữ màu đỏ, nhấn vào link màu xanh (ví dụ: `home_screen.dart:25`) để nhảy đúng đến dòng gây lỗi

🔗 Tạo lỗi CỐ Ý

`Text(task!)`
(khi `task` null)

Quan sát

- App crash
- Màn hình đỏ

✓ CHECKPOINT

Sinh viên phải xác định được: **Tên lỗi** là gì? Nó xảy ra ở **dòng bao nhiêu**?

- ✓ Biết:
 - Lỗi dòng nào
 - File nào
 - Vì sao lỗi

📌 Bài học

Lỗi không đáng sợ – không hiểu lỗi mới đáng sợ

📌 BƯỚC 9: ĐỌC LOG DEBUG

Mục tiêu: Giúp sinh viên biết cách "bắt bệnh" thông qua các dòng chữ màu đỏ trong hệ thống thay vì đoán mò.

🔗 Thao tác:

1. Gây lỗi cố ý (Nếu đã xóa lỗi ở bước 8):

- Mở file `home_screen.dart`.
- Trong danh sách `tasks`, hãy thử gọi một phần tử không tồn tại. Ví dụ, trong hàm `build`, hãy thêm dòng: `print(tasks[100]);` (Giả sử danh sách chưa có đến 101 phần tử).

2. Mở bảng Debug Console:

- Quan sát thanh công cụ dưới cùng của Android Studio.
- Nhấn vào tab **Run** hoặc **Debug Console**.

3. Phân tích nội dung lỗi:

- Tìm dòng chữ màu đỏ có nội dung: `RangeError (index): Index out of range.`
- **Tìm "Dấu vết" (Stack Trace):** Kéo lên trên một chút trong phân log, tìm các dòng chữ màu xanh bắt đầu bằng `package:my_daily_planner/...` hoặc `home_screen.dart:xx`.
- Nhấn chuột vào đường dẫn đó (ví dụ: `home_screen.dart:45`). Android Studio sẽ đưa con trỏ chuột đến đúng dòng code bị lỗi trong file của bạn

✓ CHECKPOINT

✓ Biết đọc:

Null check operator used on a null value

- ✓ Sinh viên xác định được **Loại lỗi** (Exception Type).
- ✓ Sinh viên xác định được **Vị trí lỗi** (Tên file và số dòng).
- ✓ Hiểu rằng: Khi app bị "văng" hoặc hiện màn hình đỏ, câu trả lời luôn nằm ở tab **Run/Debug Console**.

7 TỔNG KẾT WORKFLOW ĐÚNG

- ✓ flutter doctor → trước khi hoảng
- ✓ Emulator chạy → rồi mới run
- ✓ Hot Reload → UI
- ✓ Hot Restart → logic / state
- ✓ Debug từ log → không đoán mò

8 LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Cách xử lý
Emulator không hiện	Restart Emulator
flutter run fail	flutter doctor
UI không đổi	Hot Restart
Lỗi đỏ	Đọc dòng đầu tiên

9 SAU DỰ ÁN NÀY, SINH VIÊN:

- Không còn sợ setup
- Tự debug lỗi cơ bản
- Hiểu workflow Flutter chuẩn
- Sẵn sàng học:
 - Layout nâng cao
 - State management

 ĐỊNH HƯỚNG CHUNG CHO SINH VIÊN

-  KHÔNG học Dart bằng cách đọc
-  KHÔNG viết Dart như Java / C++
-  VIẾT → CHẠY → SAI → SỬA → HIỂU

 Mỗi dự án:

- Nhỏ
- Độc lập
- Chạy được bằng **Terminal**
- Kiểm tra kết quả **rõ ràng – nhìn là biết đúng/sai**

 CHUẨN BỊ MÔI TRƯỜNG (BẮT BUỘC – LÀM 1 LẦN) Bước 0: Cài VS Code & Dart 1 Mở VS Code 2 Extensions (Ctrl + Shift + X)

Tìm & cài:

- **Dart**
- **Flutter** (nếu có)

 3 Kiểm tra Dart trong Terminal

Mở Terminal trong VS Code (Ctrl + `)

`dart --version` CHECKPOINT

-  Hiện phiên bản Dart
-  Không nhận lệnh → Flutter/Dart chưa cài đúng

Bước 1: Mở Command Palette

- Nhấn tổ hợp phím: `Ctrl + Shift + P` (Windows) hoặc `Cmd + Shift + P` (Mac).
- Gõ chữ: **flutter** vào ô tìm kiếm.

Bước 2: Chọn loại dự án

- Chọn dòng: **Flutter: New Project**.
- Tiếp theo, chọn: **Application**.

Bước 3: Chọn nơi lưu dự án

- Một cửa sổ thư mục sẽ hiện ra. Bạn hãy chọn một thư mục trên máy tính (Ví dụ: `D:/FlutterProjects`) và nhấn **Select a folder to create the project in**.

Bước 4: Đặt tên dự án

- Nhập tên dự án của bạn (ví dụ: `my_new_app`).
- **Lưu ý:** Tên phải viết thường, không dấu, cách nhau bằng dấu gạch dưới `_`.
- Nhấn **Enter** và đợi khoảng 30 giây để Flutter tự động sinh code.

3. Cách chạy dự án vừa tạo trên VS Code

Sau khi dự án hiện ra, bạn thực hiện các thao tác sau để chạy app:

1. **Mở file code:** Mở file `lib/main.dart`.
2. **Chọn thiết bị (Device Selector):**
 - Nhìn xuống **góc dưới cùng bên phải** của thanh trạng thái VS Code.
 - Bạn sẽ thấy tên thiết bị hiện tại (ví dụ: `Windows (desktop)` hoặc `No Device`).
 - Nhấn vào đó để chọn máy ảo (Emulator) hoặc trình duyệt Chrome.
3. **Chạy App:**
 - Nhấn phím **F5** để chạy chế độ Debug (có hỗ trợ Hot Reload).
 - Hoặc mở Terminal (`Ctrl + ~`) và gõ: `flutter run`.

DỰ ÁN THỰC HÀNH 3

Dart Types & Variables – Quản lý thông tin cá nhân

Mục tiêu

- Hiểu **int**, **String**, **bool**, **List**, **Map**
- Phân biệt **final** vs **const**
- Chạy Dart **KHÔNG** cần Flutter UI

BƯỚC 1: CHUẨN BỊ MÔI TRƯỜNG VS CODE

Mục tiêu: Đảm bảo VS Code đã hiểu ngôn ngữ Dart.

1. Mở **VS Code**.
2. Nhấn biểu tượng **Extensions** (hình 4 ô vuông bên trái) hoặc nhấn `Ctrl + Shift + X`.
3. Tìm và cài đặt Extension: **Dart**.
4. Chọn **File -> Open Folder ->** Tạo thư mục mới tên `dart_basic_lab` và nhấn **Select Folder**.

BƯỚC 2: TẠO FILE VÀ CẤU TRÚC DỰ ÁN

1. Nhấn vào biểu tượng **New File** (hình tờ giấy có dấu cộng) ở cột bên trái. Hoặc Chuột phải → New File

- 1.
2. Đặt tên chính xác là: **main.dart**.

 **CHECKPOINT:** Nếu thấy biểu tượng logo Dart (hình khiên xanh) bên cạnh tên file là bạn đã làm đúng.

BƯỚC 3: VIẾT CODE (RẤT CƠ BẢN)

```
void main() {  
  // 1. Kiểu dữ liệu cơ bản  
  String name = "Phuc";      // Chuỗi văn bản  
  int age = 20;               // Số nguyên  
  bool isStudent = true;     // Đúng/Sai (Boolean)  
  
  // 2. Kiểu danh sách & Tập hợp
```

```

List<String> skills = ["Dart", "Flutter"]; // Danh sách các chuỗi
Map<String, int> scores = {                // Cặp Khóa : Giá trị
    "Math": 8,
    "Programming": 9
};

// 3. Phân biệt Khai báo hằng số
final String school = "FPT University"; // final: Giá trị được xác định khi chạy
app
const String country = "Vietnam";        // const: Giá trị cố định ngay khi viết
code

// 4. Xuất dữ liệu ra Terminal
print("--- Thông tin cá nhân ---");
print("Tên: $name - Tuổi: $age"); // Sử dụng $ để in biến trong chuỗi (String
Interpolation)
print("Sinh viên: ${isStudent ? 'Có' : 'Không'}");
print("Kỹ năng: ${skills.join(', ')}");
print("Điểm Programming: ${scores['Programming']}");
print("Trường: $school, Quốc gia: $country");
}

```

▶ BƯỚC 4: CHẠY CHƯƠNG TRÌNH

Cách 1: Dùng Terminal (Chuyên nghiệp)

1. Mở Terminal trong VS Code bằng phím tắt: **Ctrl + ~** (phím cạnh số 1).
2. Gõ lệnh: `dart run main.dart` và nhấn Enter.

Cách 2: Dùng nút Run (Nhanh)

1. Nhìn lên phía trên hàm `void main()`, bạn sẽ thấy chữ **Run** nhỏ màu xám.
2. Nhấn vào đó, kết quả sẽ hiện ở tab **Debug Console**.

✓ CHECKPOINT

- ✓ Terminal in ra toàn bộ dữ liệu
- ✓ Không lỗi đỏ

⚠ LỖI THƯỜNG GẶP

Lỗi	Nguyên nhân
Không chạy được	Chưa mở đúng folder
Lỗi dấu ;	Quên dấu chấm phẩy
Lỗi type	Gán sai kiểu

🔍 BƯỚC 5: THỰC HÀNH LỖI ĐỂ HIỂU BẢN CHẤT (DEBUG PRACTICE)

Hãy yêu cầu sinh viên thử các hành động sau và quan sát gạch đỏ:

1. **Lỗi kiểu dữ liệu:** Thử gán `age = "Hai mươi"`; -> Dart sẽ báo: *A value of type 'String' can't be assigned to a variable of type 'int'.*
2. **Lỗi hằng số:** Thử gán lại `country = "USA"`; -> Dart sẽ báo: *Constant variables can't be assigned a value.*

-  Hiểu rằng mỗi dòng lệnh Dart phải kết thúc bằng dấu chấm phẩy ;.
-  Biết cách dùng dấu \$ để in biến bên trong dấu ngoặc kép.
-  Phân biệt được: **int** (số), **String** (chữ), **bool** (đúng/sai).

BÀI HỌC

Dart **rất nghiêm ngặt về kiểu dữ liệu**

Điều này giúp app **ít lỗi runtime** hơn

DỰ ÁN THỰC HÀNH 4

Null Safety – Quản lý hồ sơ sinh viên

Mục tiêu

- Hiểu ký tự ? (Cho phép null) và ! (Ép buộc không null).
- Luyện tập tư duy "Kiểm tra trước khi dùng" để tránh lỗi App bị dừng đột ngột.
- Hiểu ?, !
- Tránh lỗi **null crash**

CODE

BƯỚC 1: TẠO FILE THỰC HÀNH

1. Trong thư mục `dart_basic_lab` đã tạo ở dự án 3, tạo file mới tên: `null_safety_lab.dart`.
2. Mở Terminal và sẵn sàng gõ lệnh `dart run null_safety_lab.dart`.

BƯỚC 2: VIẾT CODE VÀ QUAN SÁT (TỪNG GIAI ĐOẠN)

Giai đoạn 1: Khai báo biến có thể Null

Dart

```
void main() {  
  // Thêm dấu ? để nói với Dart: "Biến này có thể để trống (null)"  
  String? email;
```

```
  print("Email hiện tại là: $email"); // Kết quả: null
```

Giai đoạn 2: Xử lý an toàn (Cách làm đúng)

Dart

```
email = "phuc@gmail.com";
```

```
// Kiểm tra trước khi sử dụng để tránh lỗi  
if (email != null) {  
  print("Độ dài email: ${email.length}");  
} else {  
  print("Email đang trống, không thể lấy độ dài!");  
}
```

Giai đoạn 3: Toán tử Null-aware (Làm code gọn hơn)

Dart

```
// Nếu email null, dùng giá trị mặc định là "Chưa có email"
String displayEmail = email ?? "Chưa có email";
print("Thông báo: $displayEmail");
}
```

✗ BƯỚC 3: TẠO LỖI CÓ Ý (QUAN TRỌNG)

Để sinh viên hiểu sự nguy hiểm của dấu `!`, hãy yêu cầu các em thực hiện:

1. Gán lại `email = null;`
2. Viết dòng lệnh: `print(email!.length);`
3. **Quan sát:** Terminal sẽ báo lỗi đỏ rực: `Null check operator used on a null value.`

✓ CHECKPOINT: PHÂN BIỆT NHANH

Dạy sinh viên quy tắc "Bàn tay" để ghi nhớ:

Ký hiệu	Tên gọi	Ý nghĩa	Cách nhớ
<code>?</code>	Nullable	Có thể có hoặc không.	"Có thể rỗng không?"
<code>!</code>	Bang operator	Thề là không null!	"Tôi thề, nếu sai thì app crash."
<code>??</code>	If-null	Nếu null thì dùng cái này.	"Phương án dự phòng."

✓ CHECKPOINT

✓ Hiểu:

- Khi nào dùng `?`
- Khi nào **KHÔNG ĐƯỢC** dùng `!`

🎓 BÀI HỌC

✗ `!` không phải để dùng cho tiện

✓ Chỉ dùng khi **chắc chắn không null**

🎓 BÀI HỌC "XƯƠNG MÁU"

- **Không dùng `!` một cách bừa bãi:** Nhiều bạn mới học hay thêm `!` chỉ để hết gạch đỏ của VS Code. Điều này giống như việc "nhắm mắt đi đại qua đường".
- **Luôn ưu tiên `if (variable != null) hoặc ??`:** Đây là cách các lập trình viên chuyên nghiệp bảo vệ ứng dụng của họ khỏi bị văng (crash).

🌟 THỬ THÁCH NHỎ (CHALLENGE)

Yêu cầu sinh viên viết thêm:

- Tạo biến `int? score;`
- In ra dòng chữ: "Điểm của bạn là: 0" nếu `score` đang null bằng cách sử dụng toán tử `??`.

 Mục tiêu

- Biết cách khai báo và gọi hàm (Function).
- Phân biệt tham số truyền vào (Arguments) và giá trị trả về (Return value).
- Hiểu tại sao cần tách nhỏ mã nguồn để dễ quản lý.

 BƯỚC 1: TẠO FILE THỰC HÀNH

1. Trong VS Code, tạo file mới: `function_lab.dart`.
2. Mở sẵn Terminal bằng phím tắt `Ctrl + ~`.

 BƯỚC 2: VIẾT CODE (TỪ CƠ BẢN ĐẾN NÂNG CAO)

Hãy hướng dẫn sinh viên viết hàm theo 2 cách để các em thấy sự tiện lợi của Dart:

Cách 1: Khai báo hàm truyền thông

Dart

```
void main() {  
  // Gọi hàm và lưu kết quả vào biến  
  double mathScore = 8.5;  
  double progScore = 9.0;  
  
  double avg = calculateAverage(mathScore, progScore);  
  
  print("--- BẢNG ĐIỂM ---");  
  print("Điểm trung bình: $avg");  
  
  // Gọi hàm kiểm tra xếp loại  
  print("Xếp loại: ${getRank(avg)}");  
}  
  
// Hàm tính trung bình cộng (nhận vào 2 số double, trả về 1 số double)  
double calculateAverage(double a, double b) {  
  return (a + b) / 2;  
}
```

Cách 2: Hàm rút gọn (Arrow Function) Hướng dẫn sinh viên cách viết nhanh cho các hàm chỉ có 1 dòng lệnh (Rất phổ biến trong Flutter).

Dart

```
// Cách viết ngắn gọn dùng ký hiệu =>  
String getRank(double score) => score >= 5 ? "ĐẠT" : "KHÔNG ĐẠT";
```

BƯỚC 3: THỰC HÀNH THAM SỐ TÙY CHỌN (OPTIONAL PARAMETERS)

Đây là kiến thức cực kỳ quan trọng vì Flutter sử dụng nó ở mọi nơi. Hãy yêu cầu sinh viên viết thêm hàm có ngoặc nhọn {}:

Dart

```
// Hàm in thông báo có tham số đặt tên (Named Parameters)
void printInfo({required String name, double? gpa}) {
  print("Sinh viên: $name - GPA: ${gpa ?? 'Chưa có'}");
}

// Gọi hàm trong main()
void main() {
  // ... code cũ ...
  printInfo(name: "Phuc", gpa: 8.5);
  printInfo(name: "An"); // Không truyền gpa vẫn chạy được nhờ Null Safety
}
```

CHECKPOINT: KIỂM TRA TƯ DUY

Hỏi sinh viên 3 câu hỏi nhanh sau khi chạy code:

1. Tại sao hàm `calculateAverage` dùng kiểu `double` mà không dùng `int`? (Vì phép chia thường ra số thập phân).
2. Từ khóa `return` dùng để làm gì? (Để trả kết quả về nơi gọi hàm).
3. Sự khác biệt khi thêm dấu {} vào tham số là gì? (Khi gọi hàm phải viết tên tham số, giúp code dễ đọc hơn).

BÀI HỌC "VÀNG" CHO FLUTTER

- **Mỗi hàm chỉ nên làm một việc:** Một hàm chỉ nên tính điểm, một hàm chỉ nên in ấn. Đừng gộp tất cả vào một.
- **Tái sử dụng:** Bạn chỉ cần viết hàm `calculateAverage` một lần và có thể dùng cho hàng nghìn sinh viên khác nhau.
- **Sự liên quan đến Flutter:** Sau này khi làm UI, mỗi Widget thực chất cũng giống như một function nhận dữ liệu vào và trả về giao diện.

THỬ THÁCH (CHALLENGE)

Yêu cầu sinh viên viết một hàm tên là `greet` nhận vào một `String name` và trả về chuỗi "Chào bạn, [name]!". Sau đó in kết quả ra màn hình.

Tiếp theo: Bạn đã sẵn sàng để sinh viên tiến tới dự án 6 về **Hướng đối tượng (OOP) - Class & Object** chưa? Đây là bài học "khó nhằn" nhất nhưng lại là nền tảng của mọi Widget trong Flutter!



Mục tiêu

- Hiểu **Class** (Lớp): Là một "bản thiết kế" để tạo ra các đối tượng.
- Hiểu **Constructor** (Hàm khởi tạo): Cách nạp dữ liệu vào khi tạo đối tượng.
- Biết cách đóng gói thông tin và hành động (Method) vào một đơn vị duy nhất.

 BƯỚC 1: TẠO FILE THỰC HÀNH

1. Trong VS Code, tạo file mới: `oop_lab.dart`.
2. Mở sẵn Terminal để chuẩn bị chạy lệnh `dart run oop_lab.dart`.



BƯỚC 2: VIẾT CODE (TỪ CẤU TRÚC ĐẾN THỰC THI)

Hãy hướng dẫn sinh viên khai báo Class nằm **ngoài** hàm `main()` để thấy sự phân tách rõ ràng:

1. Khai báo Class và Constructor:

Dart

```
class Student {
  // 1. Các thuộc tính (Fields)
  String name;
  int age;
  double? gpa; // Có thể null

  // 2. Constructor (Cách viết rút gọn của Dart)
  // this.name có nghĩa là: Lấy giá trị truyền vào gán ngay cho biến name của Class
  Student(this.name, this.age, {this.gpa});

  // 3. Phương thức (Method) - Hành động của đối tượng
  void showInfo() {
    print("--- Thông tin sinh viên ---");
    print("Họ tên: $name");
    print("Tuổi: $age");
    print("GPA: ${gpa ?? 'Chưa cập nhật'}");
  }
}
```

2. Sử dụng Class trong hàm main():

Dart

```
void main() {
  // Tạo đối tượng (Instance) thứ nhất
  Student s1 = Student("Phuc", 20, gpa: 3.5);

  // Tạo đối tượng thứ hai
  Student s2 = Student("An", 19);

  // Gọi hành động của đối tượng
  s1.showInfo();
  s2.showInfo();
}
```

BƯỚC 3: THỰC HÀNH "SỬA ĐỔI" (DATA UPDATE)

Hướng dẫn sinh viên cách truy cập và thay đổi thuộc tính của một đối tượng đã tạo:

Dart

```
void main() {  
    Student s1 = Student("Phuc", 20);  
  
    // Cập nhật tuổi của Phuc  
    s1.age = 21;  
  
    print("Sau khi sinh nhật:");  
    s1.showInfo();  
}
```

CHECKPOINT: KIỂM TRA TƯ DUY

Hỏi sinh viên để xác nhận mức độ hiểu bài:

1. **Class và Object khác nhau thế nào?** (Class là khuôn mẫu/bản thiết kế, Object là cái bánh/sản phẩm thực tế được tạo ra từ khuôn đó).
2. **Từ khóa `this` có ý nghĩa gì?** (Dùng để trỏ đến chính thuộc tính của Class đó, tránh nhầm lẫn với tham số truyền vào).
3. **Tại sao Flutter lại cần Class?** (Mỗi màn hình, mỗi nút bấm đều cần một bản thiết kế riêng để quản lý dữ liệu và cách nó hiển thị).

BÀI HỌC "NỀN TẢNG" CHO FLUTTER

- **Mối liên hệ:** Khi bạn viết `class HomeScreen extends StatefulWidget`, bạn đang dùng chính kiến thức OOP này để xây dựng giao diện app.
- **Đóng gói:** Class giúp bạn gom nhóm dữ liệu (tên, tuổi) và hành động (in thông tin) vào một chỗ, giúp code không bị rời rạc.

THỬ THÁCH (CHALLENGE)

Yêu cầu sinh viên nâng cấp Class `Student`:

1. Thêm một phương thức tên là `updateGPA(double newGpa)` để cập nhật điểm cho sinh viên.
2. Tạo thêm một Class `Teacher` với các thuộc tính `name` và `subject`, sau đó in thông tin ra màn hình.

Bước tiếp theo: Sau khi đã nắm vững Class, sinh viên đã sẵn sàng quay trở lại Android Studio để làm dự án 7 - **Xây dựng giao diện danh sách nâng cao (ListView)** chưa? Đây là nơi Class `Student` này sẽ biến thành các Widget thực tế trên màn hình!

Mục tiêu

Dự án thực hành 7 tập trung vào **Control Flow (Luồng điều khiển)**. Đây là "bộ não" giúp ứng dụng của bạn biết khi nào nên hiển thị cái này, khi nào nên lặp lại cái kia. Trong Flutter, chúng ta sử dụng `if` và `for` rất nhiều để render danh sách Widget.

Dưới đây là hướng dẫn chi tiết từng thao tác trên **VS Code**:

DỰ ÁN THỰC HÀNH 7: Control Flow – Danh sách công việc

Mục tiêu bài học

- Sử dụng vòng lặp `for` để duyệt qua các phần tử trong danh sách.
- Sử dụng câu lệnh `if...else` để xử lý logic điều kiện.
- Hiểu cách kết hợp giữa dữ liệu và logic điều hướng.

BƯỚC 1: TẠO FILE THỰC HÀNH

1. Mở **VS Code**, tại thanh bên trái (Explorer), chuột phải chọn **New File**.
2. Đặt tên file: `control_flow_lab.dart`.
3. Mở sẵn cửa sổ **Terminal** (Ctrl + ~).

BƯỚC 2: VIẾT CODE VÀ PHÂN TÍCH

Hãy yêu cầu sinh viên gõ đoạn code sau để làm quen với cách xử lý logic:

Dart

```
void main() {  
  // 1. Khai báo danh sách  
  List<String> tasks = ["Học bài", "Code dự án", "Đi ngủ", "Tập thể  
dục"];  
  
  print("--- DANH SÁCH CÔNG VIỆC ---");  
  
  // 2. Sử dụng vòng lặp for-in để duyệt danh sách  
  for (var task in tasks) {  
  
    // 3. Sử dụng cấu trúc điều kiện if-else  
    if (task.contains("Code")) {  
      // Nếu công việc có chữ "Code" thì in kèm ký hiệu ưu tiên  
      print("🔥 Quan trọng: $task");  
    } else if (task == "Đi ngủ") {  
      // Điều kiện cụ thể  
      print("😴 Nghỉ ngơi: $task");  
    } else {  
      // Các trường hợp còn lại  
      print("📌 Thường: $task");  
    }  
  }  
  
  // 4. Thử nghiệm với toán tử điều kiện rút gọn (Ternary operator)  
  int taskCount = tasks.length;  
  String message = taskCount > 3 ? "Hôm nay khá bận!" : "Hôm nay rảnh  
rỗi."  
  print("\nThông báo: $message");  
}
```

BƯỚC 3: CHẠY CHƯƠNG TRÌNH

1. Trong Terminal, gõ: `dart run control_flow_lab.dart`.
2. **Quan sát kết quả:**
 - o Các dòng chữ hiện ra theo đúng thứ tự.
 - o Chữ "Code dự án" có biểu tượng 🔥, chữ "Đi ngủ" có biểu tượng 😴.
 - o Dòng thông báo cuối cùng hiện "Hôm nay khá bận!" vì có 4 công việc (>3).

BƯỚC 4: THỰC HÀNH SỬA ĐỔI (LOGIC TEST)

Yêu cầu sinh viên thực hiện thay đổi sau để hiểu sâu hơn:

1. Thêm lệnh `if (tasks.isEmpty)` ở đầu hàm `main` để kiểm tra danh sách có rỗng hay không.
2. Thử xóa bớt các phần tử trong `tasks` chỉ còn 2 phần tử và chạy lại lệnh `dart run`.
 - o **Kết quả mong muốn:** Dòng thông báo cuối cùng phải đổi thành "Hôm nay rảnh rỗi."

CHECKPOINT: TỔNG KẾT KỸ THUẬT

Dạy sinh viên quy tắc ghi nhớ:

- **for:** Dùng để "đi qua" từng món đồ trong một chiếc hộp (List).
- **if:** Dùng để "kiểm tra" món đồ đó có thỏa mãn yêu cầu hay không.
- **Flutter Tip:** Trong Flutter UI, bạn có thể dùng `for` trực tiếp trong mảng `children`: `[]` của một `Column` để hiển thị hàng loạt `Widget`.

BÀI HỌC THỰC TẾ

Sau này khi làm App, bạn sẽ dùng logic này để:

- **if:** Nếu người dùng chưa đăng nhập thì hiện nút "Login", nếu đã đăng nhập thì hiện "Avatar".
- **for:** Duyệt danh sách tin nhắn từ server để vẽ lên màn hình chat

DỰ ÁN THỰC HÀNH 8

`async / await` – Giả lập tải dữ liệu

Mục tiêu

- Hiểu khái niệm **Future**: Một giá trị sẽ có trong tương lai.
- Sử dụng bộ đôi **async / await** để xử lý các công việc tốn thời gian.
- Phân biệt giữa chạy tuần tự và chạy bất đồng bộ.

BƯỚC 1: TẠO FILE THỰC HÀNH

1. Trong VS Code, tạo file mới tên: `async_lab.dart`.
2. Mở sẵn Terminal bằng phím tắt `Ctrl + ~`.

BƯỚC 2: VIẾT CODE GIẢ LẬP MẠNG

Hãy hướng dẫn sinh viên gõ đoạn code này và giải thích ý nghĩa của lệnh "chờ":

Dart

```
// 1. Hàm giả lập việc tải dữ liệu từ Server
// Future<String> nghĩa là: Hàm này sẽ trả về một chuỗi String trong
// tương lai
Future<String> fetchUserData() async {
  print("Đang kết nối với Server...");

  // Giả lập độ trễ mạng 3 giây
  await Future.delayed(Duration(seconds: 3));

  return "Dữ liệu người dùng: Nguyễn Văn A";
}
```

```
// 2. Hàm main cũng phải có 'async' để sử dụng được 'await'
void main() async {
  print("--- BẮT ĐẦU TẢI APP ---");

  print("Đang gọi API...");

  // await: Dừng lại ở đây, đợi khi nào có dữ liệu mới chạy tiếp dòng
  // dưới
  String data = await fetchUserData();

  print("Kết quả: $data");
  print("--- KẾT THÚC ---");
}
```

BƯỚC 3: CHẠY VÀ QUAN SÁT HIỆN TƯỢNG

1. Gõ lệnh: `dart run async_lab.dart`.
2. **Quan sát:**
 - o Dòng "Đang gọi API..." hiện ra ngay lập tức.
 - o Chương trình "**khựng**" lại khoảng 3 giây (đây là lúc lệnh `await` đang làm việc).
 - o Sau 3 giây, dòng "Kết quả..." mới xuất hiện.

BƯỚC 4: THỰC HÀNH LỖI PHỔ BIẾN (QUÊN AWAIT)

Hãy yêu cầu sinh viên thử xóa chữ `await` trước hàm `fetchUserData()` và chạy lại:

- **Kết quả:** Biến `data` sẽ in ra là Instance of 'Future<String>' thay vì nội dung văn bản.
- **Bài học:** Nếu không có `await`, Dart sẽ lấy "tờ giấy hẹn" (Future) chứ không lấy được "món hàng" thực tế.

CHECKPOINT: TỔNG KẾT KỸ THUẬT

Dạy sinh viên quy tắc "3 Phải":

1. Muốn dùng **await** -> hàm bao bọc **phải** có `async`.
2. Hàm tốn thời gian -> kiểu trả về **phải** nằm trong `Future<...>`.
3. Muốn lấy giá trị thực từ Future -> **phải** có `await`.

TỔNG KẾT CHUỖI DỰ ÁN DART CHO SINH VIÊN

Sau khi hoàn thành từ Dự án 3 đến Dự án 8, sinh viên đã có "chứng chỉ" nhỏ về tư duy lập trình Dart:

-  **Dùng VS Code + Terminal:** Không còn phụ thuộc vào nút nhấn, hiểu bản chất dòng lệnh.
-  **Dart không còn mơ hồ:** Nắm rõ Kiểu dữ liệu, Null Safety, OOP và Async.
-  **Sẵn sàng quay lại Flutter UI:** Bây giờ khi thấy `FutureBuilder` hay `onPressed: () async {}`, sinh viên sẽ thấy quen thuộc.
-  **Tự tin sửa lỗi:** Biết đọc Debug Console để truy tìm dòng code lỗi.

BƯỚC TIẾP THEO: ÁP DỤNG VÀO FLUTTER UI

Sau bài này, bạn nên cho sinh viên quay lại dự án **Student Task Manager** để:

1. Sử dụng **Class** để quản lý danh sách Task (Thay vì String đơn thuần).
2. Dùng **async/await** để giả lập việc lưu Task vào database (hiện Loading trước khi thêm thành công).
3. Sử dụng **ListView.builder** để tối ưu hiệu năng hiển thị.

 DỰ ÁN TỔNG HỢP 9
STUDENT TASK MANAGER APP
(Flutter UI + Dart Fundamentals)

1 TỔNG QUAN DỰ ÁN

◆ Tên dự án

Student Task Manager

◆ Mô tả

Ứng dụng Flutter đơn giản cho phép sinh viên:

- Nhập tên công việc
- Thêm vào danh sách
- Đánh dấu hoàn thành
- Xóa công việc

👉 Dữ liệu **chỉ lưu trong bộ nhớ (state cục bộ)** – đúng phạm vi Module 2.

◆ Mục tiêu học tập

Sau dự án, sinh viên:

- Hiểu **tư duy widget & widget tree**
- Sử dụng **Scaffold + UI composition**
- Dùng **StatefulWidget + setState**
- Áp dụng **Dart: List, class, function, if**
- Viết code **rõ ràng – dễ đọc – đúng chuẩn**

◆ Đối tượng phù hợp

- Sinh viên mới học Flutter
- Sinh viên còn yếu thao tác
- Chưa học state management nâng cao

2 CHỨC NĂNG CHÍNH & KIẾN THỨC ÁP DỤNG

Chức năng	Kiến thức
Hiển thị danh sách công việc	List, ListView
Thêm công việc	TextField, setState
Đánh dấu hoàn thành	bool, if
Xóa công việc	List.remove
UI rõ ràng	Scaffold, Column, Row

3 CÔNG NGHỆ & CÔNG CỤ

- Flutter SDK
- Dart
- **Android Studio**
- Android Emulator
- Material Widgets

4 CẤU TRÚC THƯ MỤC DỰ ÁN

```
lib/  
├── main.dart           // Entry point  
├── models/  
│   └── task.dart      // Class Task  
├── screens/  
│   └── home_screen.dart
```

👉 Giải thích

- `models/` → chứa class Dart (logic)
- `screens/` → chứa UI màn hình
- `main.dart` → khởi động app

5 DANH SÁCH CÁC BƯỚC (TÓM TẮT)

1. Tạo project Flutter
2. Chạy app mặc định
3. Tạo cấu trúc thư mục
4. Tạo class Task
5. Tạo HomeScreen (StatefulWidget)
6. Dùng Scaffold xây UI
7. Tạo TextField nhập dữ liệu
8. Thêm task vào danh sách
9. Hiện thị danh sách bằng ListView
10. Đánh dấu & xóa task
11. Kiểm tra & sửa lỗi

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

◆ BƯỚC 1: KHỞI TẠO DỰ ÁN (ANDROID STUDIO)

1. Mở **Android Studio** -> **New Flutter Project**.
2. Chọn đường dẫn Flutter SDK -> **Next**.
3. **Project name:** `student_task_manager` (Lưu ý: Không dùng viết hoa, không dấu cách).
4. Nhấn **Finish** và đợi dự án khởi tạo xong.
5. **Thao tác dọn dẹp:** Mở `lib/main.dart`, xóa toàn bộ code mặc định và dán code sau:

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart'; // Sẽ báo đỏ cho đến khi làm Bước 3

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Task Manager',
      theme: ThemeData(primarySwatch: Colors.blue, useMaterial3: true),
      home: const HomeScreen(),
    );
  }
}
```

◆ BƯỚC 2: TẠO DATA MODEL (CLASS TASK)

Mục tiêu: Định nghĩa cấu trúc dữ liệu cho một công việc.

1. Tại thư mục `lib`, chuột phải chọn **New** -> **Directory** -> đặt tên `models`.
2. Chuột phải vào `models`, chọn **New** -> **Dart File** -> đặt tên `task`.
3. Viết code cho `task.dart`:

```

class Task {
  String title;
  bool isDone;

  // Constructor sử dụng named parameters với giá trị mặc định là false
  Task({required this.title, this.isDone = false});
}

```

◆ BƯỚC 3: TỔ CHỨC THƯ MỤC & FILE UI

1. Tại thư mục lib, chuột phải chọn **New -> Directory** -> đặt tên screens.
2. Chuột phải vào screens, chọn **New -> Dart File** -> đặt tên home_screen.

◆ BƯỚC 4: XÂY DỰNG GIAO DIỆN HOME SCREEN (STATEFULWIDGET)

Mở file home_screen.dart và bắt đầu dựng khung. Đây là nơi chứa logic thay đổi dữ liệu.

```

import 'package:flutter/material.dart';
import '../models/task.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // 1. Khai báo danh sách Task rỗng
  final List<Task> _tasks = [];

  // 2. Controller để lấy dữ liệu từ ô nhập
  final TextEditingController _controller = TextEditingController();

  // 3. Hàm thêm Task mới
  void _addTask() {
    String text = _controller.text.trim();
    if (text.isNotEmpty) {
      setState(() {
        _tasks.add(Task(title: text));
        _controller.clear(); // Xóa chữ sau khi thêm
      });
      FocusScope.of(context).unfocus(); // Ẩn bàn phím
    }
  }

  @override
  Widget build(BuildContext context) {
    // Tính toán số task hoàn thành cho Bước 8
    int completedCount = _tasks.where((t) => t.isDone).length;

    return Scaffold(
      appBar: AppBar(
        title: const Text("Student Task Manager"),
        backgroundColor: Colors.blueAccent,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),

```

```

child: Column(
  children: [
    // Thống kê nhanh
    Text("Hoàn thành: $completedCount / ${_tasks.length}"),
    const SizedBox(height: 10),

    // Ô nhập liệu (TextField)
    TextField(
      controller: _controller,
      decoration: InputDecoration(
        labelText: "Nhập tên công việc...",
        border: const OutlineInputBorder(),
        suffixIcon: IconButton(
          icon: const Icon(Icons.add),
          onPressed: _addTask,
        ),
      ),
    ),
    const SizedBox(height: 20),

    // Danh sách hiển thị (ListView)
    Expanded(
      child: _tasks.isEmpty
        ? const Center(child: Text("Chưa có công việc nào!"))
        : ListView.builder(
            itemCount: _tasks.length,
            itemBuilder: (context, index) {
              return _buildTaskItem(_tasks[index], index);
            },
          ),
    ),
  ],
),
);
}

```

◆ BƯỚC 5: TỐI ƯU HÓA ITEM HIỂN THỊ (LISTTILE)

Thêm hàm `_buildTaskItem` ngay bên dưới hàm `build` trong lớp `_HomeScreenState`. Chúng ta sử dụng `ListTile` để có bố cục chuẩn.

```

Widget _buildTaskItem(Task task, int index) {
  return Card(
    elevation: 2,
    child: ListTile(
      leading: Checkbox(
        value: task.isDone,
        onChanged: (bool? value) {
          setState(() {
            task.isDone = value!;
          });
        },
      ),
      title: Text(
        task.title,
        style: TextStyle(
          decoration: task.isDone ? TextDecoration.lineThrough : null,
          color: task.isDone ? Colors.grey : Colors.black,
        ),
      ),
    ),
  );
}

```

```

    ),
    trailing: IconButton(
      icon: const Icon(Icons.delete, color: Colors.redAccent),
      onPressed: () {
        setState(() {
          _tasks.removeAt(index);
        });
      },
    ),
  ),
);
}
} // Kết thúc _HomeScreenState

```

◆ BƯỚC 6: GIẢI THÍCH CÁC WIDGET "SÓNG CÒN"

Giảng viên cần giải thích cho sinh viên 2 điểm mấu chốt:

1. **Expanded bao quanh ListView:** Vì Column muốn chiếm hết chiều dọc, và ListView cũng muốn chiếm hết chiều dọc -> Gây ra lỗi (Crash). Expanded giúp ListView chỉ chiếm phần còn lại của màn hình.
2. **setState():** Nếu không có hàm này, danh sách `_tasks` bên trong code vẫn tăng lên nhưng Flutter sẽ không vẽ lại màn hình để người dùng thấy.

◆ BƯỚC 7: XỬ LÝ LỖI & DEBUG (HOT RELOAD)

- **Lỗi "Gạch đỏ" ở tên file:** Di chuột vào tên file báo lỗi -> Nhấn **Alt + Enter** -> Chọn **Import library**.
- **Lỗi "Overflow":** Nếu bàn phím che mất ô nhập liệu, hãy hướng dẫn sinh viên bọc Column trong một `SingleChildScrollView` hoặc dùng `Expanded` hợp lý.

◆ BƯỚC 8: KIỂM TRA & TỔNG KẾT (CHECKPOINT)

Trước khi kết thúc, sinh viên phải tự kiểm tra được:

1. [] Nhập chữ "Học Flutter" -> Nhấn nút (+) -> Thấy xuất hiện ở dưới.
2. [] Nhập trống -> Nhấn nút (+) -> Không có gì xảy ra (Tránh lỗi logic).
3. [] Tích vào Checkbox -> Chữ bị gạch ngang.
4. [] Nhấn biểu tượng thùng rác -> Task biến mất khỏi màn hình.

9 BÀI TẬP NÂNG CAO (CHO SINH VIÊN KHÁ)

- Thêm một nút **"Xóa tất cả"** ở góc màn hình.
- Hiện thị thông báo (SnackBar) khi người dùng xóa một công việc.
- Đổi màu nền của Card thành màu xanh nhạt khi task đã hoàn thành.

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

1 Scaffold & cấu trúc màn hình trong dự án nhiều screen

Bạn đang tham gia một dự án Flutter có hơn 15 màn hình, mỗi màn hình do developer khác nhau phụ trách. Một số màn hình dùng Scaffold, một số không.

Hãy phân tích ảnh hưởng của việc **không thống nhất sử dụng Scaffold** đến UI, khả năng maintain và mở rộng (SnackBar, Drawer, BottomNavigationBar).

Vì sao cách code “chạy được” trong bài tập nhỏ lại trở thành rủi ro trong dự án lớn?

Nếu bạn là người review code, bạn sẽ đề xuất quy ước gì cho team?

2 Widget Tree phức tạp và khả năng debug UI

Trong một app nội bộ, một màn hình Dashboard có Widget Tree lồng nhau rất sâu (Scaffold → Column → Row → Column → Container → ...).

UI bị lỗi spacing và overflow nhưng không có error rõ ràng.

Hãy phân tích vì sao **không hiểu Widget Tree khiến debug UI gần như bế tắc**.

Bạn sẽ tiếp cận việc phân tích và refactor widget tree này như thế nào để chuẩn bị cho layout nâng cao?

3 StatelessWidget vs StatefulWidget trong màn hình “nửa tĩnh – nửa động”

Một màn hình Profile hiển thị thông tin người dùng (tĩnh) và có nút bật/tắt chế độ chỉnh sửa (động).

Nếu toàn bộ màn hình được viết bằng StatefulWidget, điều gì sẽ xảy ra khi project mở rộng?

Ngược lại, nếu tách quá nhỏ thành nhiều StatelessWidget, có rủi ro gì?

Hãy đánh giá cách chia widget hợp lý và liên hệ đến tư duy clean UI architecture.

4 setState và giới hạn của state cục bộ

Trong một app MVP, sinh viên sử dụng `setState()` cho mọi thay đổi UI (counter, toggle, form input).

Khi app tăng lên nhiều màn hình và nhiều state, hiện tượng gì sẽ xuất hiện về mặt code và hiệu năng?

Hãy phân tích vì sao `setState()` phù hợp cho Module 2 nhưng **không phải giải pháp lâu dài**.

Điều này gợi mở nhu cầu học gì ở các module sau?

5 Column / Row và vấn đề responsive trên nhiều thiết bị

Một UI được thiết kế bằng Column và Row hiển thị đúng trên emulator Pixel 4 nhưng bị vỡ trên màn hình nhỏ.

Hãy phân tích nguyên nhân gốc rễ từ tư duy layout cơ bản.

Vì sao việc chỉ “chỉnh lại padding” không giải quyết triệt để vấn đề?

Từ tình huống này, bạn rút ra yêu cầu gì cho việc học layout nâng cao (Flex, Expanded, Responsive)?

6 Hot Reload, Hot Restart và workflow làm việc nhóm

Trong team, có developer báo rằng “Flutter không reload UI” dù code đã thay đổi.

Hãy phân tích sự khác nhau giữa Hot Reload và Hot Restart trong bối cảnh làm việc nhóm và dự án dài hạn.

Việc hiểu sai cơ chế này gây ảnh hưởng gì đến tốc độ phát triển và debugging?

Bạn sẽ đề xuất workflow debug như thế nào cho junior developer?

7 Lỗi môi trường và rủi ro khi bỏ qua flutter doctor

Một dự án Flutter chạy tốt trên máy A nhưng lỗi trên máy B dù code giống hệt nhau.

Hãy phân tích vì sao **lỗi môi trường** là rủi ro lớn trong dự án thật nhưng thường bị sinh viên bỏ qua.

Nếu là Tech Lead, bạn sẽ thiết lập checklist môi trường như thế nào để giảm lỗi không cần thiết?

Liên hệ đến tư duy “code đúng nhưng app vẫn lỗi”.

8 Dart Types & Null Safety trong UI hiển thị dữ liệu

Một màn hình hiển thị dữ liệu từ API, nhưng đôi lúc app crash do null value.

Sinh viên dùng `dynamic` hoặc `!` để “cho chạy nhanh”.

Hãy phân tích rủi ro dài hạn của cách làm này trong dự án lớn.

Null safety trong Dart giúp gì cho Flutter UI và tư duy code an toàn?

9 Control Flow & Functions ảnh hưởng đến UI như thế nào?

Một màn hình có nhiều điều kiện hiển thị UI (loading, empty, error, success).

Nếu toàn bộ logic if/else được viết trực tiếp trong `build()` thì điều gì sẽ xảy ra khi UI phức tạp hơn?

Hãy đánh giá lợi ích của việc tách logic bằng function so với viết trực tiếp trong widget tree.

Điều này chuẩn bị tư duy gì cho clean code và clean architecture?

10 Async / Await & Exceptions trong trải nghiệm người dùng

Một app sinh viên gọi API trong `initState()` nhưng UI bị đứng hoặc crash khi API lỗi.

Hãy phân tích vai trò của `async/await` và `exception handling` trong việc đảm bảo UI ổn định.

Vì sao xử lý lỗi không chỉ là vấn đề kỹ thuật mà còn là trải nghiệm người dùng?

Từ đây, sinh viên cần chuẩn bị kiến thức gì cho state management và data layer?

TỔNG KẾT ĐỊNH HƯỚNG

Sau khi trả lời 10 prompt này, sinh viên **không chỉ biết “làm được”**, mà bắt đầu:

- Nhận ra **giới hạn của Flutter Fundamentals**
- Hiểu vì sao **code học tập $\neq$ code sản xuất**
- Có tư duy chuẩn bị cho:
 - Layout nâng cao
 - State management
 - Clean Architecture Flutter

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

1 Scaffold trong ứng dụng nhiều màn hình & nhiều team

Một ứng dụng enterprise có hơn 30 màn hình, mỗi màn hình do developer khác nhau xây dựng. Một số màn hình dùng Scaffold riêng, một số lồng Scaffold vào widget con.

Phân tích rủi ro dài hạn của việc **dùng Scaffold không nhất quán** trong codebase lớn.

Vì sao cách làm “chạy được” trong app nhỏ trở thành vấn đề nghiêm trọng khi scale?

Nếu là Tech Lead, bạn sẽ thiết kế guideline Scaffold như thế nào cho team?

2 Widget Tree quá sâu và chi phí maintain

Một màn hình Dashboard có widget tree lồng sâu hơn 10 cấp để xử lý layout.

UI bắt đầu khó debug, khó chỉnh spacing và dễ overflow.

Phân tích nguyên nhân gốc rễ từ tư duy widget cơ bản.

So sánh giữa “viết nhanh cho xong” và “tách widget có chiến lược” trong dự án dài hạn.

3 StatelessWidget vs StatefulWidget trong UI thay đổi liên tục

Trong một app quản lý, nhiều màn hình có UI thay đổi theo quyền user, trạng thái dữ liệu, và tương tác.

Nếu developer dùng StatefulWidget cho toàn bộ màn hình, điều gì sẽ xảy ra khi app mở rộng?

Ngược lại, nếu quá lạm dụng StatelessWidget + `setState` rải rác, rủi ro là gì?

Hãy đánh giá cách phân chia widget hợp lý trong môi trường chuyên nghiệp.

4 setState và giới hạn khi UI ngày càng phức tạp

Một app MVP ban đầu dùng `setState()` cho mọi thay đổi UI.

Khi app tăng lên nhiều màn hình và logic phức tạp, code trở nên khó đọc và khó debug.

Phân tích vì sao `setState()` phù hợp cho giai đoạn học cơ bản nhưng không bền vững cho production.

Từ đây, sinh viên cần chuẩn bị tư duy gì cho state management nâng cao?

5 Column / Row và vấn đề responsive trong app thật

Một UI được xây dựng hoàn toàn bằng Column và Row hiển thị tốt trên một số thiết bị nhưng vỡ layout trên thiết bị khác.

Phân tích vì sao tư duy layout cơ bản dễ dẫn đến UI không responsive.

Vì sao việc “chỉnh padding cho vừa” không phải giải pháp dài hạn?

Tình huống này gợi mở kỹ thuật layout nào cần học tiếp?

6 Tách widget và ảnh hưởng đến hiệu suất

Một developer tách UI thành rất nhiều widget nhỏ để “cho sạch code”.

Tuy nhiên, UI render chậm hơn và khó theo dõi luồng dữ liệu.

Phân tích trade-off giữa **clean code** và **hiệu suất / độ rõ ràng**.

Khi nào nên tách widget, khi nào không nên trong dự án lớn?

7 Dart Types & dynamic trong codebase enterprise

Trong một dự án lớn, nhiều developer sử dụng `dynamic` để xử lý nhanh dữ liệu từ API.

Ban đầu code chạy ổn, nhưng sau đó xuất hiện lỗi runtime khó truy vết.

Phân tích vì sao việc lạm dụng `dynamic` là rủi ro nghiêm trọng trong production.

Dart typed system giúp kiểm soát chất lượng code như thế nào?

8 Null Safety và lỗi UI khó debug

Một màn hình đôi lúc crash khi dữ liệu từ API trả về null.

Developer dùng toán tử `!` để “ép chạy”.

Phân tích hậu quả dài hạn của cách làm này trong dự án lớn.

Null safety trong Dart giúp thay đổi tư duy code UI an toàn như thế nào?

9 Functions & Control Flow viết trực tiếp trong build()

Một màn hình có nhiều trạng thái UI (loading, empty, error, success).

Logic if/else được viết trực tiếp trong `build()` khiến code dài và khó đọc.

Phân tích vì sao cách làm này không phù hợp cho team lớn.

So sánh với cách tách logic bằng function và widget con.

10 Async / Await và trải nghiệm người dùng

Một app gọi API trong `initState()` nhưng UI bị đứng hoặc hiển thị sai trạng thái khi mạng chậm.

Phân tích vai trò của `async/await` trong việc đảm bảo UI mượt và ổn định.

Nếu xử lý `async` sai, hậu quả đối với UX và debugging là gì?

Điều này liên hệ thế nào đến kiến trúc data layer sau này?

1.1 Exception handling và độ ổn định của app

Một app production bị crash do exception không được catch khi gọi API.

Phân tích vì sao xử lý exception không chỉ là vấn đề kỹ thuật mà là vấn đề sản phẩm.

So sánh cách xử lý lỗi “cho xong” vs xử lý có chiến lược trong UI Flutter.

Sinh viên cần thay đổi tư duy gì khi làm app thật?

1 2 Import, tổ chức file và maintain codebase lớn

Một codebase Flutter có hàng trăm file nhưng không có quy ước import và folder rõ ràng. Developer mới mất nhiều thời gian để hiểu luồng UI. Phân tích vì sao tổ chức file là kỹ năng bắt buộc chứ không phải “tùy chọn”. Liên hệ đến clean architecture và scale team.

1 3 Hot Reload và sai lầm trong workflow chuyên nghiệp

Developer liên tục dựa vào Hot Reload để kiểm tra UI nhưng không hiểu khi nào cần Hot Restart. Dẫn đến bug “không tái hiện được”. Phân tích rủi ro của việc hiểu sai workflow debug trong team lớn. Bạn sẽ đề xuất quy trình debug chuẩn như thế nào?

1 4 Setup môi trường và rủi ro trong team nhiều người

Cùng một codebase nhưng app chạy tốt trên máy này, lỗi trên máy khác. Phân tích vì sao lỗi môi trường là nguyên nhân phổ biến gây chậm tiến độ dự án. Nếu là Tech Lead, bạn sẽ thiết lập quy trình setup & kiểm tra môi trường ra sao? Bài học gì cho sinh viên mới đi làm?

1 5 Từ Flutter Fundamentals đến kiến trúc chuyên nghiệp

Sau Module 2, sinh viên có thể xây UI và viết logic cơ bản. Phân tích giới hạn của kiến trúc này khi bước vào dự án enterprise. Những vấn đề nào **chưa thể giải quyết** chỉ với Scaffold, setState và Dart cơ bản? Từ đây, lộ trình học tiếp hợp lý cho Flutter developer chuyên nghiệp là gì?

KẾT LUẬN SỰ PHẠM

15 prompt này giúp sinh viên:

- Nhận ra **khoảng cách giữa code học tập và code production**
- Hiểu **trade-off kỹ thuật** thay vì chỉ “làm cho chạy”
- Chuẩn bị tư duy cho:
 - Advanced Layout
 - State Management (Provider, Riverpod, Bloc)
 - Clean Architecture Flutter